



МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»
РТУ МИРЭА

Институт перспективных технологий и индустриального программирования
Кафедра индустриального программирования

РАБОТА ДОПУЩЕНА К ЗАЩИТЕ

Заведующий
кафедрой _____ А.В. Юдин

«_____» _____ июня _____ 2025 г.

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА
по направлению подготовки бакалавров 09.04.02 Информационные системы и
технологии

На тему: Разработка информационной системы управления малым рестораном

Обучающийся _____
подпись

Страхов Дмитрий Сергеевич
Фамилия, имя, отчество

Шифр 22Т0986
Группа ЭФБО-06-22

Руководитель
работы _____
подпись

Ученая степень, ученое звание,
должность

Фамилия, имя, отчество

Москва 2026 г.



МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение высшего
образования

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Институт перспективных технологий и индустриального программирования

Кафедра индустриального
программирования

СОГЛАСОВАНО

УТВЕРЖДАЮ

Заведующий
кафедрой _____
подпись

Директор
института _____
подпись

Юдин Александр Викторович

Фамилия Имя Отчество

«_____» _____ 20____ г.

Пушкин Павел Юрьевич

Фамилия Имя Отчество

«_____» _____ 20____ г.

ЗАДАНИЕ

на выполнение выпускной квалификационной работы бакалавра

Обучающийся	<u>Страхов Дмитрий Сергеевич</u> <i>Фамилия Имя Отчество</i>	
Шифр	<u>22T0986</u>	
Направление подготовки	<u>09.03.02</u> <i>код направления</i>	<u>Информационные системы и технологии</u> <i>наименование направления</i>
Группа	<u>ЭФБО-06-22</u>	

1. Тема выпускной квалификационной работы

Разработка информационной системы управления малым рестораном.

2. Цель и задачи выпускной квалификационной работы

Цель работы: Разработка информационной системы для автоматизации процессов управления малым рестораном, обеспечивающей повышение эффективности обслуживания посетителей, учёта блюд и ингредиентов, а также взаимодействия с персоналом кухни и зала.

Задачи работы:

1. Исследование специфики деятельности малого ресторана как объекта автоматизации, определение ключевых проблем и потребностей в информатизации процессов управления.
2. Изучение рынка программных решений для ресторанного бизнеса, проведение их сопоставительного анализа и обоснование необходимости создания собственной

системы.

3. Описание и формирование бизнес-процессов малого ресторана в текущем и целевом состояниях посредством построения диаграмм.
4. Определение перечня требований к проектируемой системе, выработка архитектурных и проектных решений для её построения.
5. Проектирование архитектуры, интерфейса пользователя и базы данных информационной системы.
6. Реализация информационной системы с основными функциональными модулями.
7. Написание тестов, выполнение тестирования созданного программного продукта, оценка результатов, составление рекомендаций по развертыванию и сопровождению.

3. Этапы выпускной квалификационной работы

№ этапа	Содержание этапа выпускной квалификационной работы	Результат выполнения этапа ВКР	Срок выполнения
1	Обоснование актуальности работы и подготовка введения	Введение	06.03.2026
2	Исследование специфики малого ресторана, изучение и сопоставительный анализ существующих программных решений для ресторанного бизнеса, постановка задачи.	Глава 1	20.03.2026
3	Описание и построение диаграмм бизнес-процессов и формирование требований к разрабатываемому решению, проектирование архитектуры, схемы базы данных и интерфейса пользователя.	Глава 2	03.04.2026
4	Программная реализация информационной системы, тестирование, оценка результатов и составление рекомендаций по развертыванию и сопровождению.	Глава 3	17.04.2026
5	Формирование заключения и окончательное оформление выпускной квалификационной работы	Заключение, итоговый вариант ВКР	22.05.2026
6	Подготовка доклада и презентации к защите	Доклад, презентация	29.05.2026

4. Перечень разрабатываемых документов и графических материалов:

- пояснительная записка выпускной квалификационной работы;
- презентация;
- приложения.

5. Руководитель выпускной квалификационной работы

Функциональные обязанности	Должность в Университете	Фамилия Имя Отчество	Подпись
Руководитель ВКР	Доцент	Громов Евгений Владимирович	

Задание выдал

Руководитель ВКР: _____

подпись

Задание принял к исполнению

Обучающийся: _____

подпись

« _____ » _____ 20__ г.

« _____ » _____ 20__ г.

Аннотация

В данной работе рассмотрена разработка информационной системы управления малым рестораном. Проведён анализ предметной области и существующих программных решений, выявлены их недостатки и требования к системе. Также было выполнено моделирование всех процессов ресторана, были разработаны функциональные и информационные модели.

Во второй главе была спроектирована архитектура системы, которая включает серверную часть и клиентское приложение. Реализовано программное решение, которое позволяет обеспечивать управление заказами, меню, клиентами и аналитикой.

В третьей главе проведено тестирование системы, которое подтверждает корректность работы и соответствие требованиям, которые были выявлены в ходе анализа. Практическая ценность работы заключается в возможности внедрения системы для автоматизации деятельности малого ресторана, что в будущем способствует повышению эффективности управления и качества обслуживания клиентов.

Оглавление

Аннотация.....	4
Введение	7
ГЛАВА 1. ИССЛЕДОВАНИЕ СПЕЦИФИКИ ДЕЯТЕЛЬНОСТИ МАЛОГО РЕСТОРАНА КАК ОБЪЕКТА АВТОМАТИЗАЦИИ	9
1.1. Характеристика малого ресторана как объекта автоматизации	9
1.2. Современное состояние автоматизации процессов в сфере общественного питания. Обзор существующих систем	11
1.3. Аргументация в пользу создания нового программного решения в сфере общепита	13
1.4. Методические подходы к разработке информационной системы	14
1.5. Постановка задачи на разработку информационной системы	16
1.6. Первичные требования к информационной системе	17
1.6.1. Функциональные требования.....	17
1.6.2. Нефункциональные требования.....	18
1.6. Формирование первичной документации для проектирования	19
Выводы	20
ГЛАВА 2. ПРОЕКТИРОВАНИЕ ИНФОРМАЦИОННОЙ СИСТЕМЫ УПРАВЛЕНИЯ МАЛЫМ РЕСТОРАНОМ.....	22
2.1. Основные бизнес-процессы малого ресторана.....	22
2.3. Формирование требований для информационной системы малого ресторана	25
2.4. Проектирование модели данных.	29
2.5. Конфиденциальность и защита данных.....	30
2.6. Проектирование интерфейса системы	31
2.7. Выбор концепции и архитектуры разрабатываемой системы.....	32
2.8. Выбор и обоснование технологий.....	34
Выводы	36
Глава 3. ПРОГРАММНАЯ РЕАЛИЗАЦИЯ И ТЕСТИРОВАНИЕ СИСТЕМЫ УПРАВЛЕНИЯ РЕСТОРАНОМ	37
3.1. Инициализация проекта и настройка окружения.....	37
3.2. Архитектура системы	37
3.3. Реализация модели данных	38

3.4. Реализация системы управления доступом	40
3.5. Реализация POS-интерфейса экрана кухни.	41
3.6. Реализация системы скидок.	42
Список используемых литературных источников	44
Приложения	47
Приложение А - Сравнительная таблица	47
Приложение Б – Техническое задание	47
Приложение В – Use Case диаграммы.	49
Приложение Г – Проектирование интерфейса системы.....	52
Приложение Г – Выбор технологий. Сравнительные таблицы.....	55
Приложение Д – Реализация и тестирование.	60

Введение

В условиях современной экономики малый бизнес в сфере общественного питания сталкивается с высокой конкуренцией, ростом затрат на сырье и персонал, а также необходимостью оперативного реагирования на изменения спроса.

Ручной учёт заказов, инвентаря и финансов приводит к ошибкам, потерям и снижению качества обслуживания. Особую актуальность приобретает автоматизация именно для малых ресторанов и кафе, где использование дорогих корпоративных систем экономически нецелесообразно. Исходя из этого, разработка доступной, простой и эффективной информационной системы управления является востребованной задачей, что и определило выбор темы выпускной квалификационной работы бакалавра.

Цель работы – разработать информационную систему управления малым рестораном, позволяющую автоматизировать ключевые бизнес-процессы и повысить эффективность деятельности предприятия.

Объектом исследования являются процессы управления деятельностью малого ресторана.

Предметом исследования выступают методы, модели и программные средства разработки информационной системы управления малым рестораном.

Разработанная система должна обладать практической ценностью для владельцев и менеджеров малых ресторанов и кафе, так как должна позволять существенно снизить операционные издержки и повысить качество обслуживания клиентов.

Для достижения поставленной цели предоставлены следующие задачи:

- Исследование специфики деятельности малого ресторана как объекта автоматизации, определение ключевых проблем и потребностей в информатизации процессов управления.

- Изучение рынка программных решений для ресторанного бизнеса, проведение их сопоставительного анализа и обоснование необходимости создания собственной системы.
- Описание и формирование бизнес-процессов малого ресторана в текущем и целевом состояниях посредством построения диаграмм.
- Определение перечня требований к проектируемой системе, выработка архитектурных и проектных решений для её построения.
- Проектирование архитектуры, интерфейса пользователя и базы данных информационной системы.
- Реализация информационной системы с основными функциональными модулями.
- Написание тестов, выполнение тестирования созданного программного продукта, оценка результатов, составление рекомендаций по развертыванию и сопровождению.

ГЛАВА 1. ИССЛЕДОВАНИЕ СПЕЦИФИКИ ДЕЯТЕЛЬНОСТИ МАЛОГО РЕСТОРАНА КАК ОБЪЕКТА АВТОМАТИЗАЦИИ

1.1. Характеристика малого ресторана как объекта автоматизации

Объектом исследования в данной работе выступают процессы управления деятельностью малого ресторана. Предметом – методы, модели и программные средства разработки информационной системы управления.

Понятие «малый ресторан» формируется на пересечении двух нормативных категорий. В соответствии с Федеральным законом от 24.07.2007 № 209-ФЗ «О развитии малого и среднего предпринимательства в Российской Федерации» (ст. 4), к малым предприятиям относятся организации со среднесписочной численностью работников не более 100 человек; предельный размер дохода для данной категории установлен Постановлением Правительства РФ от 04.04.2016 № 265 и составляет 800 млн рублей [2][3]. В соответствии с ГОСТ 31985-2013 «Услуги общественного питания. Термины и определения» (п. 25), ресторан определяется как предприятие питания, предоставляющее услуги по организации питания с широким ассортиментом блюд сложного изготовления, включая фирменные блюда, алкогольные и безалкогольные напитки, кондитерские и хлебобулочные изделия [4]. Совокупность указанных критериев позволяет определить малый ресторан как предприятие общественного питания ресторанного типа, соответствующее законодательным критериям малого бизнеса.

Особенностью организационной структуры малого ресторана является концентрация управленческих функций у ограниченного круга лиц. Согласно ст. 4 Закона № 209-ФЗ, малые предприятия имеют упрощённую организационную структуру [2], вследствие чего задачи планирования меню,

контроля запасов, учёта продаж и отчётности нередко выполняются одним-двумя сотрудниками (директором и менеджером). Это повышает операционную нагрузку и увеличивает вероятность ошибок при ручной обработке данных.

С технологической точки зрения ресторан сочетает в себе производственный цикл (приготовление блюд) и сервисный (обслуживание гостей). Согласно классификации автоматизируемых процессов, приведённой в отраслевой литературе, все операции ресторана делятся на фронт-офисные (приём заказа, передача на кухню, расчёт с гостем) и бэк-офисные (складской учёт, работа с поставщиками, бухгалтерия) [1]. Такое разделение порождает устойчивые информационные потоки между подразделениями: заказ, оформленный в зале, должен оперативно поступить на кухню, а данные о расходе ингредиентов – автоматически отразиться на складе [1].

Автоматизация общественного питания можно определить как применение программно-аппаратных средств для управления ключевыми процессами заведения: приёмом заказов, складским учётом, кассовыми операциями и аналитикой продаж [1]. Основу такой системы составляют модули кассы (POS), склада и управленческой отчётности, интегрированные в единую информационную среду [1].

Начальные требования к разрабатываемой информационной системе формируются исходя из всех вышеперечисленных фактов, а именно нормативное определение ресторана, законодательные критерии малого предприятия и отраслевые характеристики процессов автоматизации. Разрабатываемая система в совокупности должна охватывать приём и обработку заказов, складской учёт с автоматическим списанием по рецептурам, аналитику продаж и формирование управленческой отчётности.

1.2. Современное состояние автоматизации процессов в сфере общественного питания. Обзор существующих систем

Современный рынок автоматизации предприятий общественного питания характеризуется устойчивым развитием, обусловленным ростом конкуренции, дефицитом квалифицированного персонала и необходимостью строгого соблюдения требований федерального законодательства. В условиях постоянного повышения себестоимости сырья и аренды владельцы заведений всё чаще обращаются к цифровым решениям, которые позволяют автоматизировать приём заказов, управление складом, производство, финансы и взаимодействие с клиентами.

На рынке автоматизации общественного питания представлен широкий спектр решений – от полнофункциональных ERP-платформ до лёгких облачных POS-систем. Для выявления функциональных решений, применимых в разрабатываемой системе, проводится технический обзор и анализ трёх наиболее распространённых продуктов: iiko, R-Keeper и Quick Resto.

Система iiko построена на облачной ERP-архитектуре. Архитектурно делится на три компонента: iikoFront (фронт-офис), iikoOffice (бэк-офис) и iikoCloud (облачная аналитика и удалённый доступ). В системе реализованы следующие роли пользователей: администратор, менеджер зала, официант, кассир, бармен, технолог, кладовщик. Разграничение прав настраивается для каждой роли отдельно [9].

Документооборот системы включает: пре-чек, фискальный чек, акт приготовления, акт списания, приходную накладную, инвентаризационную ведомость, технологическую карту блюда. Аналитические отчёты: по продажам в разрезе блюд, официантов и периодов; ABC-анализ меню; отчёт по себестоимости продуктов; отчёт по движению склада; Z-отчёт кассовой смены; отчёт по персоналу. Поддерживается интеграция с ЕГАИС, «Честным

знаком», сервисами доставки (Яндекс.Еда, Delivery Club) и 1С:Бухгалтерия [10].

Полноценная функциональность складского учёта и аналитики доступна начиная с тарифа Pro, что существенно увеличивает стоимость владения для малого заведения. Внедрение системы, как правило, требует привлечения сертифицированных партнёров.

R-Keeper – одна из старейших систем автоматизации ресторанного бизнеса, разработанная компанией UCS и насчитывающая более 30 лет присутствия на рынке. Архитектура построена по распределённому принципу: каждый модуль функционирует автономно, что обеспечивает работу в офлайн-режиме без потери данных. Основные компоненты: r_keeper Front/CashDesk (фронт-офис, касса), StoreHouse (складской учёт), r_keeper Cloud (облачное решение для СМБ), KDS (кухонные дисплеи), модуль Delivery, Kiosk самообслуживания [11].

Реализованные роли: администратор, менеджер, кассир, официант, бармен, кладовщик. Система формирует: заказ, пре-чек, фискальный чек, приходную накладную, акт списания, инвентаризационную ведомость, технологическую карту, отчёт по ЕГАИС. Аналитические отчёты: по выручке, по блюдам, по операторам, по кассовым сменам (X- и Z-отчёты), по движению склада, ABC-анализ. Поддерживается интеграция с ЕГАИС, системами лояльности и сторонними доставщиками [12].

Распределённая архитектура является ключевым техническим преимуществом при работе в условиях нестабильного интернет-соединения.

Quick Resto – российская облачная система, ориентированная на сегмент малого и среднего бизнеса. Архитектурно состоит из кассового терминала и веб-бэк-офиса, доступного из браузера. Поддерживается офлайн-режим работы терминала [13].

Роли пользователей: владелец/администратор, менеджер, официант, кассир, кладовщик. Права доступа настраиваются отдельно для бэк-офиса и

кассового терминала. Документооборот: приходная накладная (в том числе через ЭДО/УПД), акт списания, акт перемещения, акт приготовления, акт разбора и переработки, инвентаризационная ведомость, фискальный чек. Аналитические отчёты: по продажам в разрезе дней, официантов, кассиров, блюд и типов оплат; Z-отчёт смены; RFM-анализ клиентской базы; генератор произвольных отчётов с настраиваемыми полями. Отчёт по персоналу доступен только на тарифах Pro и Enterprise. Интеграции: ЕГАИС, «Честный знак», «Меркурий», Яндекс.Еда, Delivery Club, DocsInBox (автоматическая загрузка накладных от поставщиков) [14].

Система полностью адаптирована к требованиям российского законодательства (54-ФЗ, ЕГАИС, маркировка). Наличие генератора произвольных отчётов выгодно отличает её от конкурентов.

Сравнительная анализ выведен в таблицу, которая находится в приложении А.

1.3. Аргументация в пользу создания нового программного решения в сфере общепита

Проведённый анализ современных отечественных и зарубежных систем автоматизации общественного питания позволяет сделать вывод о наличии существенного разрыва между предложением рынка и реальными потребностями малых ресторанов. Корпоративные решения обеспечивают высокий уровень функциональности и надёжности, однако их внедрение сопряжено со сложностью настройки и необходимостью подключения дополнительных модулей.

Если говорить о конкретных выделенных проектных решениях для разрабатываемой системы по нескольким направлениям, то стоит отметить решения, которые точно должны быть. Во-первых, двухкомпонентная архитектура «фронт-офис + веб-бэк-офис» по образцу Quick Resto. Обязательно должна быть ролевая модель с отдельной настройкой прав для

каждого, аналогично Quick Resto. Во-вторых, обязательно должны быть формирование чеков, пре-чеков, асинхронная работа заказов и обработки заказов на кухне, карта ресторана со столиками для официантов, и складской учёт.

Стоит также отметить, что следует пересмотреть отношение к генератору отчётности и отчётности по персоналу. В рассматриваемых системах такой функционал доступен только при подключении дополнительных модулей либо при покупке других тарифов. Но важно то, что для малого ресторана это не имеет смысла, удобнее сделать единый продукт, который будет удовлетворять все базовые потребности. Исходя из этого в разрабатываемой системе эти функции будут включены в основную версию. Тоже самое можно сказать и про ABC-анализ меню, в разрабатываемой системе он будет включён в стандартный набор аналитики.

1.4. Методические подходы к разработке информационной системы

Разработка информационных систем представляет собой сложный процесс, требующий выбора методологии, которая определяет структуру жизненного цикла проекта, последовательность этапов, механизмы контроля и возможности адаптации к изменениям. В научной литературе выделяют несколько классических и современных подходов, каждый из которых имеет свои преимущества и ограничения в зависимости от характера проекта, уровня неопределенности требований и масштаба бизнеса.

Каскадная модель или waterfall – одна из старейших и наиболее структурированных методологий [5]. Она предполагает строго последовательное прохождение этапов: сбор и анализ требований, проектирование, реализация, тестирование, внедрение и сопровождение. Переход к следующему этапу возможен только после полного завершения предыдущего и утверждения всех документов. Данная модель обеспечивает

высокую предсказуемость сроков и бюджета, четкую документацию и удобна для проектов с фиксированными, хорошо изученными требованиями. Однако она практически не допускает внесения изменений на поздних стадиях, что приводит к высоким рискам в случае выявления ошибок или новых потребностей заказчика.

Итерационная модель предусматривает разработку системы в повторяющихся циклах (итерациях). Каждая итерация включает мини-версию полного цикла (проектирование – реализация – тестирование – оценка) и завершается рабочим прототипом или инкрементом функционала. Это позволяет поэтапно уточнять требования, оперативно реагировать на обратную связь и минимизировать риски. Модель особенно эффективна, когда требования к системе формируются постепенно в процессе взаимодействия с пользователями.

Agile или Scrum – современный гибкий подход, основанный на принципах манифеста Agile. Разработка ведется короткими итерациями (спринтами), в рамках которых команда выдает работающий продукт. Agile идеально подходит для проектов с высокой динамикой изменений, неопределенностью требований и необходимостью постоянного вовлечения заказчика.

Для разработки информационной системы управления малым рестораном целесообразно использовать каскадную модель. В отличие от динамичных проектов с постоянно меняющимися требованиями, бизнес-процессы малого ресторана (прием и обработка заказов, управление меню, учет поставок и склада, аналитика продаж, отчетность) являются достаточно стабильными и хорошо формализуемыми уже на этапе предпроектного обследования. Каскадный подход обеспечивает строгую последовательность работ, что особенно важно для малого бизнеса с ограниченными ресурсами. Кроме того, каскадная модель гарантирует высокую степень соответствия

системы нормативным требованиям, поскольку все требования фиксируются заранее и не подвергаются частым изменениям.

1.5. Постановка задачи на разработку информационной системы

По итогам анализа предметной области и обзора существующих систем автоматизации можно сформулировать цель и задачи разрабатываемой информационной системы.

Цель разработки – информационная система управления малым рестораном, реализованная в виде веб-приложения и объединяющая фронт-офис (приём заказов, касса) и бэк-офис (склад, аналитика, управление меню).

Архитектурное решение заимствовано из практики Quick Resto, однако в отличие от него не требует установки нативного приложения на терминал – весь интерфейс, включая рабочее место официанта и кассира, работает в браузере.

В результате анализа аналогов были выявлены типовые задачи. Реализация ролевой модели из шести ролей (администратор, менеджер, официант, повар, кладовщик) с отдельной настройкой прав, опять же по образцу Quick Resto и R-Keeper. Соответственно итоговая ролевая модель состоит из шести ролей: администратор – все функции, настройка системы, пользователи; менеджер – меню, техкарты, склад, аналитика, отчёты; официант – приём заказов, работа со столами, пре-чек, расчёт с гостем, фискальный чек, касса; кладовщик – складские документы, инвентаризация, остатки; повар – просмотр очереди заказов, смена статуса блюда. Реализация автоматического списания ингредиентов по технологическим картам с расчётом себестоимости методом FIFO. Обеспечение приёма заказов с визуальной схемой зала, передачей на кухню и формированием пре-чека и фискального чека – это уже стандартные функции фронт-офиса всех рассматриваемых систем.

Задачи, которые решаются иначе – генератор отчётов и отчётность по персоналу. Включение ABC-анализа меню без дополнительных модулей. Также реализация дашборда руководителя с ключевыми показателями о выручке, среднем чеке, топе блюд, как единой страницы для роли администратора или менеджера. В рассмотренных системах аналогичная функция обычно выносится в отдельный модуль или мобильное приложение.

Данные выводы согласуются с общими положениями теории информационных систем, согласно которым внедрение специализированных программных решений позволяет повысить эффективность управления и снизить влияние человеческого фактора [1].

1.6. Первичные требования к информационной системе

На основании анализа предметной области и обзора существующих систем автоматизации сформированы функциональные и нефункциональные требования к разрабатываемой системе. Ниже представлены основные группы требований.

1.6.1. Функциональные требования

ФТ-01. Управление заказами. Система должна обеспечивать создание заказа с привязкой к столу, добавление модификаторов и комментариев к позициям, изменение состава заказа, применение скидок и промокодов, разделение счёта между гостями и автоматический пересчёт итоговой суммы.

ФТ-02. POS-интерфейс. Должны быть реализованы выбор стола из карты зала с отображением занятости, навигация по категориям меню и визуальная индикация блюд с недостаточным запасом ингредиентов.

ФТ-03. Экран кухни. Система должна отображать очередь активных заказов с цветовой индикацией времени ожидания, обеспечивать подтверждение готовности с уведомлением официанта и синхронизацию экрана при поступлении новых заказов.

ФТ-04. Карта зала. Структура зала организована иерархически: зал – зона – стол. Для каждого стола поддерживаются три статуса (свободен, занят, забронирован), статусы синхронизируются между устройствами в режиме реального времени.

ФТ-05. Управление меню. Должны поддерживаться создание, редактирование и удаление категорий, блюд и групп модификаторов, временная деактивация блюд без удаления, ведение технологических карт с нормами расхода ингредиентов.

ФТ-06. Складской учёт. Система должна вести учёт ингредиентов с указанием единицы измерения, текущего остатка, минимального уровня запаса и стоимости; поддерживать операции приёмки, списания и инвентаризации;

ФТ-07. Система скидок. Должны поддерживаться процентные и фиксированные скидки с применением на весь заказ или отдельную позицию, а также промокоды с ограничением по датам, количеству использований и статусу активности.

ФТ-08. Расчёт и печать чеков. Система должна обеспечивать выбор способа оплаты.

ФТ-10. Аудит. Система должна вести журнал критичных действий с фиксацией пользователя.

ФТ-11. Управление персоналом и доступом. Должны поддерживаться управление учётными записями и ролями, а также разграничение прав доступа по функциональным модулям.

1.6.2. Нефункциональные требования

НТ-01. Безопасность. Пароли пользователей хранятся в хэшированном виде; доступ разграничивается на основе ролей и разрешений; персональные данные защищаются в соответствии с Федеральным законом от 27.07.2006 № 152-ФЗ «О персональных данных» [21].

НТ-02. Развёртывание. Система разворачивается на одном сервере без специализированной IT-инфраструктуры, что соответствует ресурсным ограничениям малого ресторана.

1.6. Формирование первичной документации для проектирования

В рамках данной работы, которая демонстрирует разработку информационной системы управления малым рестораном можно выделить ключевой документ, а именно бриф проекта. Стоит отметить, что именно данный документ обеспечивает формализацию требований к системе и определяет основные параметры её разработки, включая функциональные возможности, ограничения [7].

Бриф представляет собой структурированное описание разрабатываемой системы, в котором фиксируются цели и назначение проекта, характеристики объекта автоматизации, а также совокупность требований.

В проекте данной работы техническое задание включает описание следующих аспектов: назначение системы, связанное с автоматизацией процессов учёта заказов, управления меню и контроля складских ресурсов; характеристика объекта автоматизации, отражающая особенности функционирования малого ресторана и существующие проблемы организации учета; функциональные требования, определяющие состав реализуемых возможностей системы, включая управление заказами, складской учет и формирование отчетности; требования к архитектуре системы, предусматривающие использование клиент-серверной модели с разделением на серверную часть и пользовательский интерфейс; требования к программной и технической среде, необходимые для функционирования системы.

Полная версия брифа проекта представлена в приложении Б. Он выступает основным инструментом формализации требований к системе и обеспечивает целостность и обоснованность последующих этапов её создания.

Выводы

Первая глава посвящена всестороннему изучению малого ресторана в качестве объекта автоматизации. Показано, что такое предприятие отличается компактными масштабами, высокой интенсивностью операций, выраженной сезонностью спроса, постоянной работой со скоропортящимися продуктами и тесной интеграцией процессов обслуживания гостей (фронт-офис) с производственными и складскими задачами (бэк-офис). Ключевые информационные потоки – от приема заказов и формирования меню до учета клиентов, персонала, поставок, складских остатков, аналитики продаж и формирования отчетности – образуют единую взаимосвязанную цепочку, где сбой в любом звене сразу сказывается на общей эффективности бизнеса. Автоматизация этих потоков дает возможность существенно уменьшить влияние человеческого фактора, сократить издержки и обеспечить быстрое принятие обоснованных управленческих решений.

Сравнительный обзор популярных систем автоматизации ресторанного бизнеса выявил их принципиальные недостатки применительно к малому формату. Технический анализ систем iiko, R-Keeper и Quick Resto показал, что все три решения закрывают типовой набор задач – ролевую модель, складской документооборот, расчёт себестоимости по FIFO и кассовые операции – однако ни одно из них не подходит малому ресторану в полной мере. iiko и R-Keeper требуют приобретения отдельных модулей для складского учёта и аналитики и привлечения сертифицированных партнёров при внедрении. Quick Resto переносит расширенную аналитику и отчётность по персоналу на платные тарифы дороже.

Анализ существующих методик разработки информационных систем подтвердил целесообразность применения каскадной модели жизненного цикла. При относительной стабильности и хорошей структуре бизнес-процессов малого ресторана такой подход гарантирует четкую последовательность этапов, а также строгое соответствие системы заранее

утвержденным требованиям. Для предприятия с ограниченными ресурсами каскадная модель позволяет избежать рисков перерасхода средств и отклонения конечного результата от реальных нужд.

Материалы первой главы убедительно обосновывают актуальность и целесообразность разработки специализированной информационной системы для автоматизации малого ресторана. Создание такой системы с использованием каскадной модели позволит преодолеть выявленные недостатки существующих решений, обеспечить полноценную поддержку всех основных бизнес-процессов.

ГЛАВА 2. ПРОЕКТИРОВАНИЕ ИНФОРМАЦИОННОЙ СИСТЕМЫ УПРАВЛЕНИЯ МАЛЫМ РЕСТОРАНОМ

2.1. Основные бизнес-процессы малого ресторана

Система ориентирована на малый ресторан с одной локацией и штатом из небольшого количества человек, работающих одновременно. Из этого вытекают два ключевых требования к проектированию: достаточно развернуть систему на одном сервере без горизонтального масштабирования, но при этом все устройства персонала должны получать обновления данных в режиме реального времени.

В системе предусмотрено четыре роли пользователей. Администратор имеет полный доступ к системе: управляет персоналом, ролями, меню, схемой зала, складом, отчётностью и журналом аудита. Официант работает через POS-терминал – принимает и ведёт заказы, видит карту зала и меню. Повар взаимодействует с кухонным экраном (KDS): видит поступающие заказы и состав блюд. Кладовщик отвечает за складской учёт – оформляет приёмку товаров, списания и проводит инвентаризацию.

Общий вид диаграммы использования отображён на Рисунке 2.1.1. Стоит пройтись по каждой роли и продемонстрировать варианты использования для каждой роли конкретно.

Диаграмма вариантов использования для роли «Официант» находится в Приложении В, рисунок 3.1, и охватывает полный цикл обслуживания гостя: от выбора стола до расчёта и печати чека. Центральный сценарий – создание и ведение заказа: выбор блюд, добавление модификаторов, изменение состава и применение скидок. Отдельно описаны сценарии расчёта с гостем, выбора способа оплаты и разделения счёта между гостями.

Диаграмма для роли «Повар» находится в Приложении В, рисунок 3.2, и описывает работу с кухонной подсистемой. Повар просматривает очередь активных заказов, контролирует время ожидания и отмечает блюда как

готовые. После смены статуса система автоматически уведомляет официанта, синхронизируя работу кухни и зала.

Диаграмма для роли «Кладовщик» находится в Приложении В, рисунок 3.3, и отражает сценарии складского учёта: оформление поступлений, списаний и инвентаризации, а также контроль текущих остатков. Система фиксирует причины списания, автоматически рассчитывает расхождения при инвентаризации и сигнализирует о низком уровне запасов.

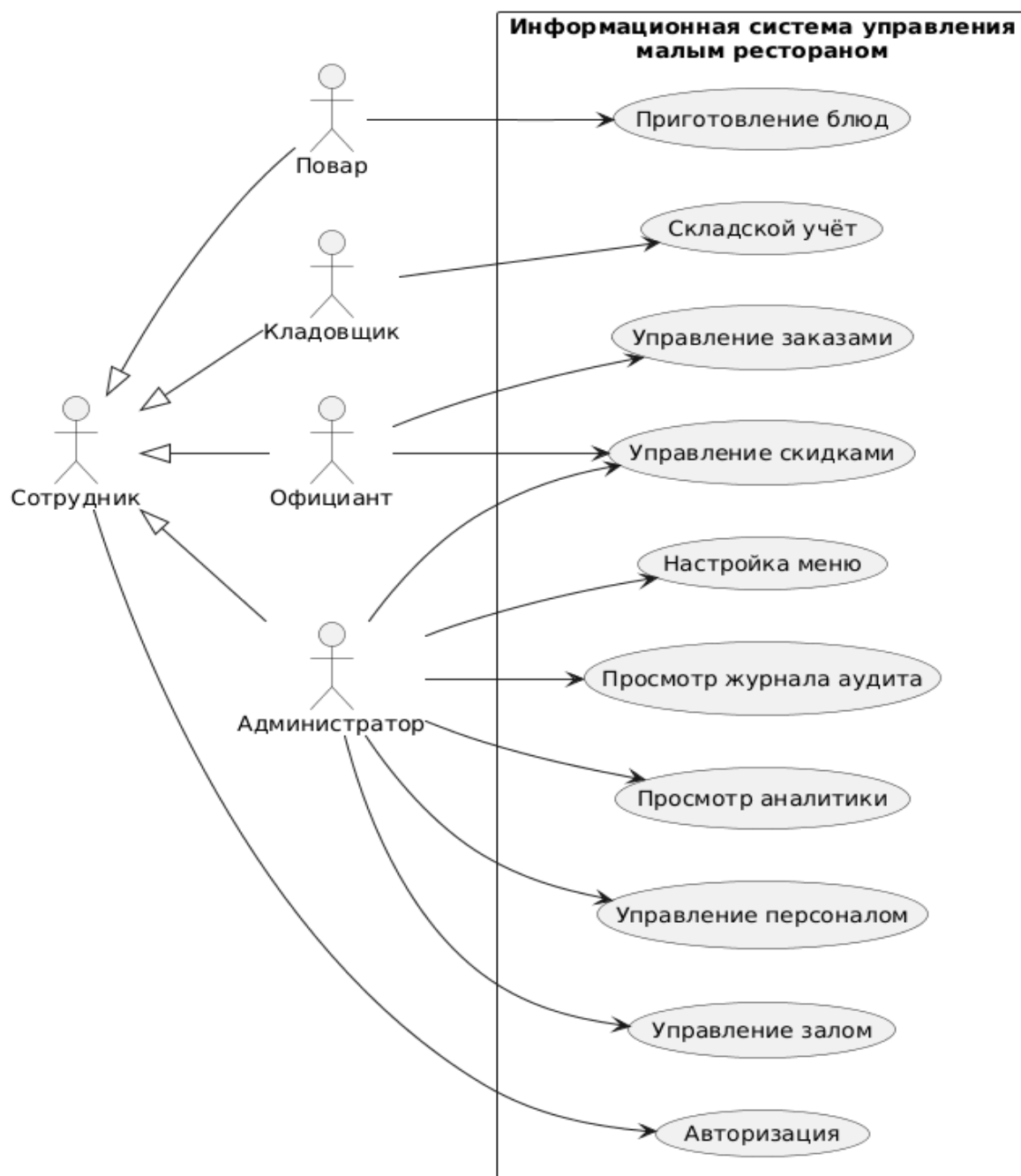


Рисунок 2.1.1 – Общая диаграмма использования системы.

Диаграмма для роли «Администратор» находится в Приложении В, Рисунок 3.4, и описывает укрупнённые сценарии управления системой: настройку персонала, меню и схемы зала, управление скидочной политикой, анализ ключевых показателей и контроль действий сотрудников через журнал аудита. Детализированные операции – создание учётных записей, назначение ролей, управление технологическими картами, модификаторами и аналитическими отчётами – реализованы в рамках соответствующих функциональных модулей.

Также стоит обратить внимание на основные жизненные циклы: заказ и складской учёт.

Центральным бизнес-процессом системы является жизненный цикл заказа, реализованный в виде конечного автомата с шестью состояниями: «Новый», «В работе», «Готов», «Подан», «Оплачен» и «Отменён» (Рисунок 2.1.2).

Переход из состояния «Новый» в «В работе» происходит автоматически в момент появления заказа на кухонном экране: повар видит новый заказ и приступает к приготовлению. По завершении приготовления повар нажимает кнопку «Готово» на KDS – заказ переходит в состояние «Готов», а событие должно транслироваться в реальном времени, и официант мгновенно получает уведомление. Далее официант фиксирует доставку блюд гостю, переводя заказ в состояние «Подан». При оплате заказ получает статус «Оплачен»: система автоматически списывает ингредиенты со склада согласно технологическим картам, формирует записи складских движений и переводит стол в статус «Свободен». Отмена заказа допустима на любом этапе, кроме состояния «Оплачен»; при отмене стол также освобождается.



Рисунок 2.1.2 – Диаграмма состояний заказа.

Складской процесс построен на четырёх типах движения ингредиентов. Приёмка увеличивает остаток и фиксирует стоимость единицы товара. Списание уменьшает остаток и требует указания причины. Корректировка формируется автоматически по итогам инвентаризации как разница между фактическим и учётным количеством. Списание по заказу выполняется автоматически при оплате и рассчитывается на основании технологических карт. Для каждого ингредиента задаётся минимальный уровень запаса. При снижении остатка ниже установленного порога система формирует предупреждение, которое отображается в POS-интерфейсе официанта рядом с соответствующими блюдами.

2.3. Формирование требований для информационной системы малого ресторана

На основании анализа бизнес-процессов сформированы следующие функциональные требования к системе.

Первое требование. Управление заказами:

- создание заказа с привязкой к столу и выбором позиций из меню;
- добавление модификаторов – как предустановленных, так и в виде свободного комментария – к отдельным позициям;
- изменение количества позиций и их удаление из текущего заказа;
- применение скидок и промокодов на уровне заказа и отдельных позиций;
- разделение счёта с переносом позиций в новый заказ;
- автоматический пересчёт итоговой суммы при каждом изменении.

Второе требование. POS-интерфейс:

- выбор стола из карты зала с отображением его текущей занятости;
- навигация по категориям меню и добавление блюд в заказ;
- визуальная индикация блюд, для которых недостаточно ингредиентов на складе.

Третье требование. Экран кухни (KDS):

- отображение очереди активных заказов в режиме реального времени;
- цветовая индикация времени ожидания: зелёный – менее 10 минут, жёлтый – менее 20 минут, красный – более 20 минут;
- кнопка подтверждения готовности с мгновенной трансляцией события;
- автоматическое обновление при поступлении новых заказов.

Четвёртое требование. Карта зала:

- иерархическая структура: залы -> зоны -> столы;
- три статуса стола: свободен, занят, забронирован;
- обновление статусов в реальном времени.

Пятое требование. Управление меню:

- создание, редактирование и удаление категорий, блюд и групп модификаторов;
- временная деактивация блюд без их удаления из системы;
- ведение технологических карт: привязка ингредиентов к блюду с нормой расхода.

Шестое требование. Складской учёт:

- учёт ингредиентов с указанием единицы измерения, текущего остатка, минимального уровня запаса и стоимости;
- оформление приёмки, списания и инвентаризации с автоматическим формированием корректировок;
- автоматическое списание ингредиентов при оплате заказа на основании технологических карт;
- полная история складских движений.

Седьмое требование. Система скидок:

- два типа скидок: процентная и фиксированная;
- область применения: на весь заказ или на отдельную позицию;
- промокоды с ограничением по датам действия, количеству использований и статусу активности.

Восьмое требование. Расчёт и печать чеков:

- выбор способа оплаты: наличные или карта;
- формирование предчека и финального чека в формате PDF;
- автоматическое списание ингредиентов и освобождение стола при проведении оплаты.

Девятое требование. Аналитика и отчёты:

- панель KPI: выручка, количество заказов, средний чек, динамика выручки по дням;
- BCG-матрица блюд по показателям популярности и маржинальности;
- отчёт по себестоимости блюд: фудкост в процентах и маржинальная прибыль;
- показатель оборачиваемости столов;
- фильтрация всех отчётов по выбранному периоду.

Десятое требование. Аудит:

- журнал критичных действий: отмена позиций и заказов, изменение цен, проведение оплаты, применение и удаление скидок, использование промокодов;
- для каждой записи фиксируются пользователь, тип действия, объект, полезная нагрузка.

Одиннадцатое требование. Управление персоналом и доступом:

- создание, редактирование и удаление учётных записей пользователей с назначением ролей;

Также сформированы нефункциональные требования: требования к безопасности, требования к надёжности, требования к сохранности и целостности данных, требования к удобству использования.

Если мы говорим про требования к безопасности, то это в первую очередь про то, что система должна обеспечивать разграничение прав доступа в зависимости от роли пользователя. Каждый сотрудник должен иметь доступ только к тем разделам и операциям, которые необходимы ему для выполнения должностных обязанностей. Данные об учетных записях, заказах, оплатах и внутренних операциях должны быть защищены от несанкционированного просмотра, изменения и удаления.

Требования к надёжности четко говорят о том, что система должна обеспечивать устойчивую работу в течение всего рабочего дня ресторана. При возникновении ошибок вводимые пользователем данные не должны теряться без предупреждения. Критически важные операции, связанные с заказами, оплатой, движением товарных остатков и изменением статусов, должны выполняться корректно и однозначно, без дублирования и противоречивых состояний.

Сохранность и целостность данных — это всегда в первую очередь обеспечение логической целостности информации о заказах, меню, остатках ингредиентов, рецептурах, скидках и пользователях. Все взаимосвязанные данные должны храниться согласованно. Удаление или изменение сущностей не должно приводить к нарушению структуры данных и потере значимой информации.

Пользовательский интерфейс должен быть интуитивно понятным для сотрудников ресторана и не требовать длительного обучения. Основные действия должны выполняться за минимальное число шагов. Элементы интерфейса должны быть логично сгруппированы в соответствии с рабочими сценариями администратора, официанта, кассира и сотрудников кухни.

2.4. Проектирование модели данных.

Информационное пространство разрабатываемой системы автоматизации предприятия общественного питания организовано в виде совокупности взаимосвязанных сущностей, которые можно разделить на семь основных предметных областей:

Топология заведения. Данный блок описывает физическую планировку и включает иерархическую структуру «зал – зона – стол». Это позволяет не только вести учет посадочных мест, но и визуализировать текущее состояние загрузки зала в реальном времени.

Каталог продукции (Меню и модификаторы). Представляет собой структурированный перечень блюд и напитков. Реализованная система модификаторов обеспечивает гибкость при формировании заказов, позволяя учитывать индивидуальные пожелания гостей.

Учет транзакций (Заказы). Центральный узел системы, который объединяет всю информацию о продажах, составе чеков, методах оплаты и операциях по разделению счетов.

Складской учет и производство. Область отвечает за автоматизацию списания ингредиентов на основе технологических карт. Включает в себя механизмы контроля остатков, регистрации приходов, инвентаризаций и внутренних перемещений.

Скидки и акции. Конфигурация скидочных правил с ограничениями по времени, количеству использований и области применения.

Администрирование и безопасность. Реализует ролевую модель управления доступом, обеспечивая разграничение прав персонала в зависимости от их должностных обязанностей.

Системное логирование (Аудит). Обеспечивает прослеживаемость действий пользователей, что необходимо для анализа внештатных ситуаций и предотвращения злоупотреблений.

2.5. Конфиденциальность и защита данных

Безопасность проектируемой системы управления рестораном разрабатывается с учётом того, что обрабатываемые данные обладают высокой коммерческой значимостью. К таким данным относятся финансовые показатели деятельности предприятия (включая выручку, средний чек и себестоимость блюд), персональные данные сотрудников (ФИО, адреса электронной почты, учётные записи), а также сведения о складских остатках, рецептурах и применяемых скидочных механизмах. Нарушение конфиденциальности, целостности или доступности указанной информации способно привести к финансовым потерям и снижению управляемости бизнес-процессов, что обуславливает необходимость комплексного подхода к обеспечению информационной безопасности.

В рамках проектирования проведён анализ характерных угроз, релевантных для рассматриваемой предметной области. К числу ключевых угроз отнесён несанкционированный доступ к системе, включая попытки подбора учётных данных. Отдельное внимание уделено рискам, связанным с превышением полномочий, при котором пользователь получает возможность выполнять операции, не предусмотренные его ролью. Существенную опасность представляют атаки, направленные на перехват пользовательских сессий, позволяющие злоумышленнику действовать от имени аутентифицированного пользователя.

Для разграничения прав доступа выбрана модель RBAC. Расшифровывается как Role-Based Access Control. Это делается для того, чтобы у каждого пользователя были свои роли, которые предоставляют ему доступ к тем или иным функциям системы. У каждой роли должны быть свои разрешения. Такая система полностью должна покрывать потребности малого ресторана, потому что в ресторане существуют чётко определённые должности с фиксированным набором полномочий, и количество ролей невелико и стабильно.

Также при проектировании учтены типовые уязвимости веб-приложений, включая межсайтовые атаки, такие как внедрение вредоносного кода через пользовательский ввод, а также выполнение несанкционированных действий посредством подложных запросов. Дополнительно рассмотрены угрозы, связанные с инъекциями, при которых злоумышленник пытается внедрить произвольные команды в запросы к базе данных с целью обхода бизнес-логики и получения доступа к данным.

Такие уязвимости требуют определённого подхода, а именно: cookie недоступен из JavaScript, что нейтрализует XSS-атаки, направленные на похищение идентификатора сессии; cookie не передаётся при межсайтовых POST-запросах, что минимизирует поверхность для CSRF-атак; при развёртывании в production cookie передаётся только по HTTPS.

2.6. Проектирование интерфейса системы

Для того чтобы интерфейс полностью соответствовал требованиям удобства пользования для работников ресторана были спроектированы схемы окон всей информационной системы. Благодаря анализу бизнес-процессов и диаграмм пользовательских сценариев можно определить, как должен выглядеть интерфейс системы для полноценной работы.

Можно начать с общего каркаса всего веб-приложения. Демонстрация находится в Приложении Г, Рисунок 1.1. Каркас демонстрирует как будут выглядеть все страницы системы.

Далее идёт панель управления или главная страница. Демонстрация находится в Приложении Г, Рисунок 1.1. На странице должны будут отображены главная статистика по продажам, блюдам с графиками.

Следующий экран – это конечно же POS-экран. Нужен для официанта. При помощи этого экрана отображаются все столы доступные в ресторане. Демонстрация находится в Приложении Г, Рисунок 1.2. Тут же стоит обратить внимание еще на экран создания заказа, где будет происходить отображение блюд и формирования чека.

Экран KDS. Экран для кухни, где отображаются все заказы. Передается информация поварам. Демонстрация находится в Приложении Г, Рисунок 1.3.

Последним основным окном является экран склада. Там отображаются складские остатки. Находится в Приложении Г, Рисунок 1.4.

2.7. Выбор концепции и архитектуры разрабатываемой системы

Выбор концепции разрабатываемой системы обусловлен требованиями, сформулированными в разделе 2.3, и особенностями предметной области. В работе ресторана одновременно участвуют несколько категорий сотрудников – официанты, повара, администраторы, кладовщики, – каждый из которых взаимодействует с системой через разные устройства: стационарные компьютеры, планшеты, терминалы на кухне. Это предъявляет требование кроссплатформенности и централизованного хранения данных.

Для систем подобного класса возможна реализация трёх основных подходов: нативные приложения, десктопные приложения и веб приложения. Нативные и десктопные решения требуют установки и обновления на каждом устройстве, что создаёт дополнительную нагрузку на сопровождение в условиях ограниченных ИТ-ресурсов малого предприятия. Веб-приложение обеспечивает доступ с любого устройства через браузер без установки клиентского программного обеспечения, что соответствует требованию НТ-06 о развёртывании без специализированной инфраструктуры. Анализ рассмотренных аналогов также подтверждает обоснованность данного выбора: Quick Resto реализует бэк-офис в виде веб-интерфейса [14], что в большинстве случаев является достаточным для управления малым заведением.

Исходя из всего вышесказанного можно прийти к выводу, что разрабатываемая система должна реализовываться в виде веб-приложения с клиент-серверной архитектурой.

При проектировании рассматривались три архитектурных стиля.

Монолитная архитектура предполагает единую кодовую базу, развёртываемую как один процесс. Достоинства подхода – простота разработки, тестирования и развёртывания. Основной недостаток – высокая связанность модулей, затрудняющая независимое масштабирование отдельных компонентов

Микросервисная архитектура основана на разделении системы на независимые сервисы, взаимодействующие через API. Преимущества – независимое масштабирование и технологическая гибкость. Недостатки – значительная инфраструктурная сложность, необходимость координации и повышенные требования к DevOps-компетенциям [17].

Модульная монолитная архитектура представляет собой единое приложение с логически изолированными модулями и чёткими границами ответственности. Подход сочетает простоту развёртывания монолита с логической структурированностью, что делает его наиболее целесообразным для систем малого и среднего масштаба [18].

Для разрабатываемой системы выбрана модульная монолитная архитектура. Выбор обоснован тремя соображениями.

Во-первых, масштаб задачи. Система рассчитана на малый ресторан с небольшим количеством персонала. Микросервисная архитектура целесообразна для крупных проектов с высокими требованиями к масштабированию; при данном масштабе она привнесла бы неоправданную инфраструктурную сложность [17].

Во-вторых, простота развёртывания. Малый бизнес, как правило, не располагает выделенной IT-службой, что делает единое приложение предпочтительным с точки зрения установки и обслуживания [17].

В-третьих, модульная организация кода. Несмотря на единую кодовую базу, система разделена на логически изолированные модули – управление заказами, меню, складской учёт, аналитика, управление залом и кухонный экран, – каждый из которых имеет собственный контроллер, модели данных и

представления. Это обеспечивает возможность сопровождать и расширять систему.

Проектируемая архитектура представлена на рисунке 2.6.1



Рисунок 2.6.1 – Архитектура проектируемой системы

2.8. Выбор и обоснование технологий

Для реализации серверной части рассматривались три фреймворка: Laravel (PHP), Django (Python) и Express.js (Node.js).

Выбран Laravel 13. Фреймворк предоставляет наиболее полный набор встроенных инструментов: WebSocket через Reverb, готовую аутентификацию (Breeze), ORM Eloquent и систему очередей. Это сокращает количество сторонних зависимостей и упрощает дальнейшее сопровождение (сравнение технологий находится в Приложении Г, таблица 4.1).

Клиентская часть могла быть реализована при помощи трёх технологий: Vue 3 + Inertia.js, React + Next.js, Blade. Выбран Vue 3 + Inertia.js v2. Inertia позволяет строить полноценное SPA без написания отдельного REST API: контроллеры Laravel передают данные напрямую в Vue-компоненты. По сравнению с подходом «API + SPA» кодовая база сокращается примерно вдвое. Vue 3 с Composition API обеспечивает реактивный интерфейс с чёткой структурой логики (сравнение технологий находится в Приложении Г, таблица 4.2).

Для реализации базы данных была выбрана PostgreSQL. Рассматривались ещё MySQL, SQLite. PostgreSQL лучше подходит для данного проекта. JSON-поля используются для хранения модификаторов заказов и полезной нагрузки аудит-лога; полноценные транзакции необходимы при одновременной работе нескольких официантов (сравнение технологий находится в Приложении Г, таблица 4.3).

WebSocket сервер был выбран Laravel Reverb. Self-hosted решение не зависит от сторонних сервисов, что важно при нестабильном интернет-соединении. Авторизация каналов реализована через стандартный механизм Laravel, что сохраняет единую модель безопасности. Другие технологии даже не рассматривались, по причине того, что Laravel поддерживает Reverb нативно.

Для тестирования выбран Pest PHP. Для сравнения были выбраны Pest PHP и PHPUnit (сравнение технологий находится в приложении 4, таблица 4.4). Синтаксис `it()/expect()` сокращает объём тестового кода на 30–40% по сравнению с PHPUnit. Встроенные архитектурные тесты позволяют автоматически проверять соблюдение архитектурных ограничений – например, что контроллеры не обращаются к базе данных напрямую.

Выводы

Во второй главе определены проектные решения, на которых строится разрабатываемая система.

В качестве архитектурного подхода выбрана модульная монолитная архитектура с клиент-серверным взаимодействием. Выбор обоснован масштабом задачи: система рассчитана на малый ресторан с небольшим числом пользователей, что делает микросервисную архитектуру избыточной по сложности и стоимости сопровождения.

Технологический стек сформирован по принципу минимизации сторонних зависимостей: Laravel 13 обеспечивает серверную логику, маршрутизацию, ORM и WebSocket через встроенный сервер Reverb; Vue 3 совместно с Inertia.js реализует реактивный интерфейс без необходимости разработки отдельного REST API; PostgreSQL выбран как СУБД с поддержкой JSON-полей и полноценных транзакций.

Проектируемая модель данных включает таблицы, организованные в семь функциональных групп: зал, меню, заказы, склад, скидки, аудит и пользователи. Структура охватывает все ключевые бизнес-процессы – от приёма заказа до списания ингредиентов и формирования аналитики.

Требования систематизированы в 11 блоков, охватывающих управление заказами, POS-интерфейс, кухонный экран, складской учёт, систему скидок, аналитику и аудит. Ролевая модель включает шесть ролей с разграничением доступа.

Глава 3. ПРОГРАММНАЯ РЕАЛИЗАЦИЯ И ТЕСТИРОВАНИЕ СИСТЕМЫ УПРАВЛЕНИЯ РЕСТОРАНОМ

3.1. Инициализация проекта и настройка окружения

Проект создан на основе стартового комплекта Laravel Breeze, который предоставляет готовую систему аутентификации – регистрацию, вход, сброс пароля и подтверждение email – с фронтенд-интерфейсом на Vue 3 и Inertia.js. Окружение разработки включает PHP 8.5 и Laravel 13 на серверной стороне, PostgreSQL в качестве основной базы данных и SQLite в оперативной памяти для изолированного запуска тестов. Сборка фронтенда выполняется через Vite с Node.js, WebSocket-сервер реализован на Laravel Reverb.

В процессе разработки одновременно запускаются три процесса: PHP-сервер (`php artisan serve`), слушатель очередей (`php artisan queue:listen`) и Vite dev-сервер с перезагрузкой модулей после внесения изменений в реальном времени.

3.2. Архитектура системы

Система будет построена по шаблону MVC (Model–View–Controller), где Inertia.js выступает связующим слоем между серверными контроллерами Laravel и клиентскими Vue-компонентами. Входящий запрос последовательно проходит через middleware, контроллер и слой валидации (FormRequest), после чего управление передаётся сервисному классу с бизнес-логикой. Сервис работает с моделями Eloquent, при необходимости инициирует события для трансляции через WebSocket и возвращает данные в Vue-компонент через `Inertia::render()`.

Контроллеры должны быть сделаны тонкими: бизнес-логика, требующая координации нескольких моделей, вынесена в отдельные

сервисные классы (Диаграмма архитектуры представлена в Приложении Г, рисунок 4.1).

3.3. Реализация модели данных

Модель данных системы для малого ресторана исходя из выявленных бизнес-моделей, а также требований, может включать 15 сущностей, сгруппированных по функциональным областям. ER-диаграмма находится в Приложении Г, рисунок 4.2.

Группа «Зал» описывает физическую структуру ресторана. Hall хранит залы, Zone – зоны внутри каждого зала, Table – столы с указанием вместимости и текущего статуса. Связи выстроены по иерархии: Hall -> 1:N -> Zone -> 1: N -> Table.

Группа «Меню» охватывает номенклатуру блюд и модификаторов. Category группирует блюда по типу (салаты, супы и т.д.), MenuItem хранит сами блюда с ценой и флагом активности. Модификаторы организованы в группы: ModifierGroup (например, «Соусы») содержит отдельные позиции Modifier с ценовой надбавкой. Связь между блюдами и группами модификаторов реализована как many-to-many через промежуточную таблицу.

Группа «Заказы» фиксирует операционную деятельность. Order привязан к столу и сотруднику, содержит статус, итоговую сумму, способ оплаты и информацию о применённой скидке. OrderItem хранит позиции заказа; поле modifiers содержит JSON-массив выбранных модификаторов. Связи: Table -> 1:N -> Order -> 1:N -> OrderItem -> N:1 -> MenuItem.

Группа «Склад» обеспечивает учёт ингредиентов. Ingredient хранит единицу измерения, текущий остаток, минимальный уровень запаса и стоимость. Recipe связывает блюда с ингредиентами и нормой расхода (технологические карты). StockMovement ведёт полную историю складских движений с указанием типа операции, количества, стоимости и причины.

Связи: MenuItem -> 1:N -> Recipe -> N:1 -> Ingredient; Ingredient -> 1:N -> StockMovement.

Группа «Скидки» представлена одной сущностью Discount с гибкой конфигурацией: тип (процентная или фиксированная), область применения, промокод, даты действия и ограничение по количеству использований.

Группа «Аудит» содержит полиморфный журнал AuditLog, где каждая запись фиксирует пользователя, тип действия, объект воздействия и JSON-payload со специфичными для действия данными.

Группа «Пользователи» построена на базе сущности User в связке с пакетом Spatie Laravel Permission, который добавляет пять служебных таблиц для хранения ролей и разрешений.

База данных создана через последовательные миграции Laravel. Каждая миграция описывает одну таблицу или изменение структуры и при необходимости может быть откатана.

Для каждой из моделей реализованы фабрики – классы, генерирующие тестовые данные через библиотеку Faker с поддержкой русского языка. Фабрика модели User поддерживает специализированные состояния для каждой роли:

Листинг 3.2.1 – PHP код фабрик

```
User::factory()->admin()->create(); // администратор
User::factory()->waiter()->create(); // официант
User::factory()->chef()->create(); // повар
```

Начальное наполнение базы данных выполняется через сидеры, которые последовательно создают: четыре роли с 25 разрешениями, учётную запись администратора, два зала с зонами и столами, категории меню с блюдами и модификаторами, более 20 ингредиентов с реалистичными остатками, технологические карты для блюд и демонстрационные заказы.

Листинг 3.2.1 – Пример seeder для создания профиля администратора.

```
$admin = User::firstOrCreate(
```

```

        ['email' => 'admin@restomanager.local'],
        [
            'name' => 'Администратор',
            'password' => bcrypt('password'),
            'email_verified_at' => now(),
        ],
    );
    $admin->assignRole('admin');

```

3.4. Реализация системы управления доступом

Ролевая модель реализована на базе пакета Spatie Laravel Permission. В системе определены 4 роли и более 25 разрешений, сгруппированных по 12 функциональным модулям.

Разграничение доступа работает на трёх уровнях. На уровне маршрутов `middleware permission`: блокирует запрос при отсутствии у пользователя необходимого разрешения. На уровне WebSocket-каналов авторизация в `routes/channels.php` проверяет разрешения в момент подключения к каналу. На уровне интерфейса `composable usePermissions()` скрывает элементы, недоступные текущему пользователю, без обращения к серверу.

Листинг 3.3.1 – Пример middleware permission.

```

Route::get('/pos', [PosController::class, 'index'])
    ->middleware('permission:orders.create')
    ->name('pos.index');

```

Листинг 3.3.2 – Пример проверки permission в channels.

```

Broadcast::channel('floor', function ($user) {
    return $user->can('floor.view');
});

```

Передача данных о правах доступа на фронтенд реализована через `middleware HandleInertiaRequests`: при каждом запросе он включает в ответ информацию о текущем пользователе, его ролях и разрешениях. Это позволяет Vue-компонентам адаптировать отображение интерфейса на основе актуальных данных без дополнительных запросов к серверу.

3.5. Реализация POS-интерфейса экрана кухни.

POS-интерфейс состоит из двух экранов.

Экран выбора стола (Pos/Index.vue) отображает залы в виде вкладок, внутри каждого зала – зоны со столами. Цвет стола соответствует его статусу: зелёный – свободен, красный – занят, жёлтый – забронирован. При выборе стола выполняется переход к экрану заказа.

Экран заказа (Pos/Order.vue) построен из трёх колонок. Левая панель содержит категории меню для навигации. В центральной области отображается сетка блюд выбранной категории с ценами; блюда с недостаточным запасом ингредиентов помечены визуальной индикацией. Правая панель представляет собой корзину текущего заказа с позициями, количеством, модификаторами, комментариями и итоговой суммой.

Создание заказа (OrderController::store) выполняется в рамках транзакции базы данных и включает пять шагов: создание записи Order с привязкой к столу и пользователю; создание записей OrderItem для каждой позиции с сохранением модификаторов в JSON-поле; пересчёт итоговой суммы заказа; изменение статуса стола на «Занят»; трансляция события OrderCreated через WebSocket.

Экран кухни (Kds/Index.vue) отображает активные заказы со статусами «Новый» и «В работе» в виде карточек. Каждая карточка содержит номер стола и имя официанта, список позиций с модификаторами и комментариями, таймер с момента создания заказа с цветовой индикацией – зелёный до 10 минут, жёлтый от 10 до 20 минут, красный свыше 20 минут – и кнопку «Готово».

Актуальность данных на экране обеспечивается двумя механизмами. Основной – подписка на каналы kds и orders через Laravel Echo: при поступлении событий OrderCreated, OrderUpdated и OrderBumped список заказов обновляется автоматически. Резервный – периодический запрос к

эндпоинту `/api/kds/orders` каждые 30 секунд на случай разрыва WebSocket-соединения.

При нажатии кнопки «Готово» контроллер `KdsController::bump` переводит заказ в статус «Готов» и транслирует событие `OrderBumped`, после чего на устройствах официантов немедленно появляется уведомление о готовности блюд.

Листинг 3.4.1 – Пример контроллера экрана кухни.

```
public function bump(Order $order): RedirectResponse
{
    if ($order->status !== OrderStatus::InProgress) {
        return redirect()->back()
            ->with('error', 'Только заказы в процессе могут быть отмечены
как готовые.');
```

```
    }

    $order->update(['status' => OrderStatus::Ready]);

    OrderBumped::dispatch($order->fresh());

    return redirect()->back()
        ->with('success', "Заказ для стола #{ $order->table->number }
готов!");
}
```

3.6. Реализация системы скидок.

Логика работы со скидками сосредоточена в сервисном классе `DiscountService`. Метод `applyToOrder()` применяет скидку на уровень всего заказа и записывает `discount_id` и `discount_amount` в модель `Order`. Метод `applyToItem()` применяет скидку к отдельной позиции. Метод `applyPromoCode()` перед применением валидирует промокод – проверяет его активность, срок действия и остаток лимита использований. Метод `calculateAmount()` рассчитывает итоговый размер скидки: для процентных – как долю от суммы заказа, для фиксированных – как константу, но не

превышающую итоговую сумму. Метод `incrementUsage()` увеличивает счётчик использований промокода после его применения.

На уровне модели `Discount` реализован метод `isValid()`, который одновременно проверяет три условия: статус активности, попадание текущей даты в период действия и наличие неисчерпанного лимита использований.

Листинг 6. Пример функции из сервиса `DiscountService`.

```
public function applyToOrder(Order $order, Discount $discount): void
{
    if ($discount->scope !== DiscountScope::Order) {
        throw ValidationException::withMessages(['discount_id' => 'Эта
скидка применяется только к позициям.']);
    }

    if (! $discount->isValid()) {
        throw ValidationException::withMessages(['discount_id' =>
'Скидка недействительна.']);
    }

    $subtotal = (float) $order->items()->sum(
        \Illuminate\Support\Facades\DB::raw('unit_price * quantity -
discount_amount')
    );

    $discountAmount = $discount->calculateAmount($subtotal);

    $order->update([
        'discount_id' => $discount->id,
        'discount_amount' => $discountAmount,
    ]);

    $order->recalculateTotal();
    $discount->incrementUsage();

    AuditLog::record(AuditAction::DiscountApplied, $order, [
        'discount_id' => $discount->id,
        'discount_name' => $discount->name,
        'discount_amount' => $discountAmount,
    ]);
}
```

Список используемых литературных источников

1. Кандаленкова А. О., Назойкин Е. А. АНАЛИЗ СОСТОЯНИЯ СИСТЕМ АВТОМАТИЗАЦИИ ОБЩЕСТВЕННОГО ПИТАНИЯ // Инновации и инвестиции. 2023. №5. URL: <https://cyberleninka.ru/article/n/analiz-sostoyaniya-sistem-avtomatizatsii-obschestvennogo-pitaniya> (дата обращения: 07.03.2026).
2. О развитии малого и среднего предпринимательства в Российской Федерации: федер. закон от 24.07.2007 № 209-ФЗ (ред. от 29.12.2025) [Электронный ресурс]. – URL: <https://normativ.kontur.ru/document?moduleId=1&documentId=504300#h923> (дата обращения: 31.03.2026).
3. О предельных значениях дохода, полученного от осуществления предпринимательской деятельности, для каждой категории субъектов малого и среднего предпринимательства: постановление Правительства Рос. Федерации от 04.04.2016 № 265 [Электронный ресурс]. – URL: https://www.consultant.ru/document/cons_doc_LAW_196415/ (дата обращения: 31.03.2026).
4. ГОСТ 30389–2013. Услуги общественного питания. Предприятия общественного питания. Классификация и общие требования [Электронный ресурс]. – Введ. 2016-01-01. – URL: <https://vsegost.com/Catalog/57/57023.shtml> (дата обращения: 31.03.2026).
5. Stepanovdandcorpinfosys. Каскадная, итерационная и спиралевидная модели внедрения корпоративных информационных систем [Электронный ресурс] // Хабр. – 02.04.2024. – URL: <https://habr.com/ru/articles/804629/> (дата обращения: 09.03.2026)
6. POS-системы для ресторана [Электронный ресурс]. – Режим доступа: https://revvy.ai/blog/pos_system_dlya_restorana_kak_vibrat – Дата обращения: 07.03.2026

7. Липаев В. В. Проблемы развития программной инженерии и некоторые фрагменты их решения // Бизнес-информатика. 2007. №1. URL: <https://cyberleninka.ru/article/n/problemy-razvitiya-programmnoy-inzhenerii-i-nekotorye-fragmenty-ih-resheniya> (дата обращения: 09.03.2026).
8. Best restaurant POS systems 2025 comparison guide [Электронный ресурс]. – Режим доступа: <https://www.superiorseating.com/blog/best-restaurant-pos-systems-2025-comparison-guide> – Дата обращения: 09.03.2026
9. Лемма. 3 скрытые возможности iiko, о которых не знают 90% рестораторов [Электронный ресурс]. – 17.09.2025. – URL: <https://lemma-group.ru/articles/iiko-vozmozhnosti/> (дата обращения: 31.03.2026).
10. MultiBit. Что такое iiko – обзор функций и возможностей системы автоматизации ресторанов [Электронный ресурс]. – URL: https://multi-bit.com/avt_iiko/iiko_chno_eto (дата обращения: 31.03.2026).
11. Штрих-Центр. R-Keeper StoreHouse – автоматизация с 1С и оборудованием [Электронный ресурс]. – URL: https://shtrih-center.ru/po_oborud/r-keeper_storehouse.html (дата обращения: 31.03.2026).
12. IT-НМ. r_keeper_7 [Электронный ресурс]. – URL: <https://www.it-hm.ru/soft/r-keeper/r-keeper-v7> (дата обращения: 31.03.2026).
13. HF. Quick Resto [Электронный ресурс]. – URL: https://hf.ru/services/quick_resto (дата обращения: 31.03.2026).
14. Startpack. Quick Resto – обзор сервиса [Электронный ресурс]. – URL: <https://startpack.ru/application/quickresto-automation#integrations> (дата обращения: 31.03.2026).
15. 7 ошибок в автоматизации ресторана, из-за которых бизнес теряет деньги [Электронный ресурс]. – Режим доступа: <https://s-bz.ru/news/7->

oshibok-v-avtomatizatsii-restorana-iz-za-kotoryh-biznes-teryaet-dengi/–

Дата обращения: 09.03.2026

16. Российский рынок цифровизации HoReCa. Обзор TAdviser [Электронный ресурс]. – Режим доступа: https://www.tadviser.ru/index.php/Статья:Российский_рынок_цифровизации_HoReCa._Обзор_TAdviser– Дата обращения: 15.03.2026
17. Селезнёв А. И., Селезнёв И. Л. Актуальность применения микросервисной архитектуры в системах обработки данных [Электронный ресурс]. – URL: <https://moluch.ru/archive/495/108462/> (дата обращения: 01.04.2026).
18. Земсков Максим Александрович ИСПОЛЬЗОВАНИЕ МОДУЛЬНЫХ МОНОЛИТОВ ДЛЯ РАЗРАБОТКИ МАСШТАБИРУЕМЫХ WEB-ПРИЛОЖЕНИЙ // Universum: технические науки. 2024. №10 (127). URL: <https://cyberleninka.ru/article/n/ispolzovanie-modulnyh-monolitov-dlya-razrabotki-masshtabiruemyh-web-prilozheniy> (дата обращения: 01.04.2026).
19. Установка Laravel (Installation) [Электронный ресурс]. – Режим доступа: <https://laravel.com/docs/13.x/installation> – Дата обращения: 15.03.2026
20. Введение в Vue (Introduction) [Электронный ресурс]. – Режим доступа: <https://vuejs.org/guide/introduction.html> – Дата обращения: 15.03.2026
21. Документация PostgreSQL (PostgreSQL Documentation) [Электронный ресурс]. – Режим доступа: <https://www.postgresql.org/docs/> – Дата обращения: 23.03.2026
22. Федеральный закон "О персональных данных" от 27.07.2006 N 152-ФЗ (последняя редакция) // КонсультантПлюс URL: https://www.consultant.ru/document/cons_doc_LAW_61801/ (дата обращения: 25.03.2026).

Приложения

Приложение А - Сравнительная таблица

Критерий	iiko	R-Keeper	Quick Resto
Архитектура	Облако + модули	Распределённая	Облако
Офлайн-режим	Частично	Полный	Терминал
Роли пользователей	Есть	Есть	Есть
Складской учёт	В составе	Отдельный модуль	В составе
Генератор отчётов	Да	OLAP (настраиваемый)	Да
ЕГАИС / маркировка	Да	Да	Да
Интеграция с 1С	Да	Да	Да
Ориентация	Средний/крупный	Средний/крупный	Малый/средний

Приложение Б – Техническое задание

Бриф

на разработку

Информационной системы управления малым рестораном

1. Общие сведения	
Наименование системы	Информационная система управления малым рестораном
Основание для разработки	Необходимость автоматизации бизнес-процессов ресторана, выявленная в ходе анализа предметной области.
2. Назначение и цели разработки	
2.1 Назначение системы	
Объект автоматизации	Учёт заказов Управление меню Складской учёт Формирование отчётности
2.2 Цель разработки	

Цель	Создание программного решения, обеспечивающего повышение эффективности управления рестораном за счёт автоматизации основных бизнес-процессов.
3. Характеристика объекта автоматизации	
Описание объекта	Малый ресторан, в котором на момент начала разработки: Обработка заказов частично выполняется вручную Отсутствует централизованный учёт склада Аналитика ведётся ограниченно
4. Требования к системе	
4.1 Функциональные требования	
Управление заказами	Создание заказа Редактирование заказа Закрытие заказа Разделение счёта
Управление меню	Добавление и удаление блюд Изменение цен Распределение по категориям
Складской учёт	Учёт ингредиентов Списание при заказе Фиксация поступлений Контроль остатков
Управление пользователями	Регистрация пользователей Разграничение прав доступа
Отчётность	Отчёты по выручке Популярные блюда Статистика заказов
4.2 Требования к интерфейсу	
UI/UX	Простой и интуитивный интерфейс Минимальное количество действий для оформления заказа
5. Требования к данным	
Хранимые сущности	Заказы Блюда Ингредиенты Пользователи

6. Состав и содержание работ	
Этапы разработки	1. Анализ требований 2. Проектирование системы 3. Разработка серверной части 4. Разработка интерфейса 5. Тестирование 6. Развёртывание

Приложение В – Use Case диаграммы.

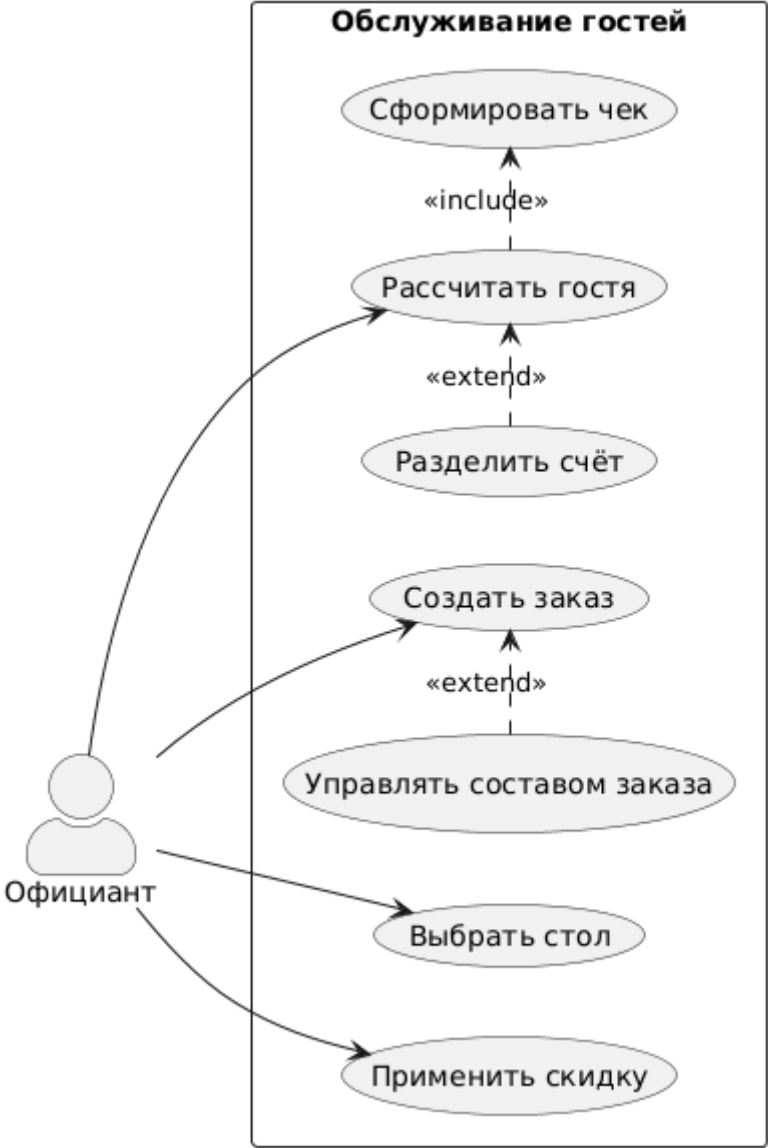


Рисунок 3.1. Use Case официант.

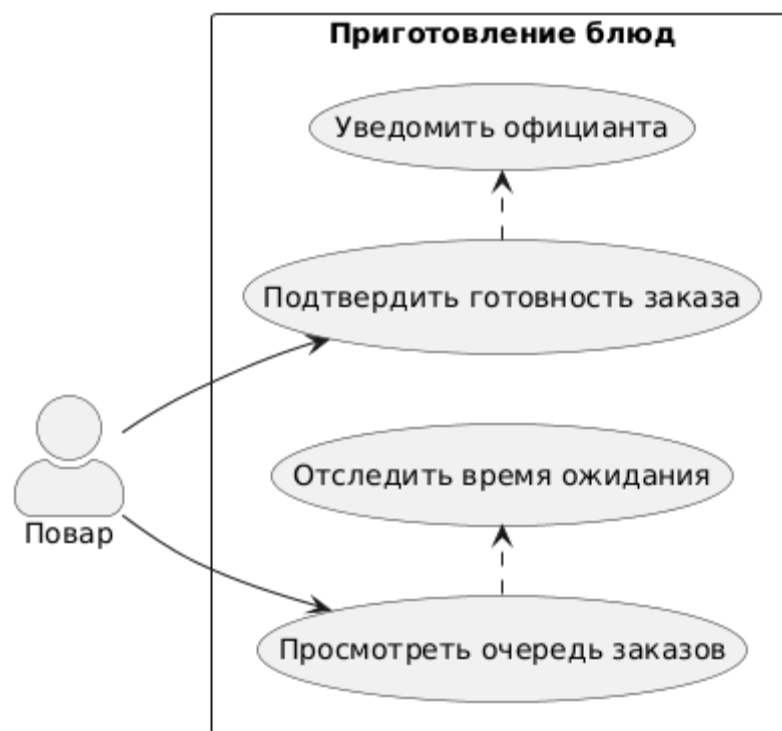


Рисунок 3.2. Use Case повар.



Рисунок 3.3. Use Case кладовщик.



Рисунок 3.4. Use Case Администратор.

Приложение Г – Проектирование интерфейса системы

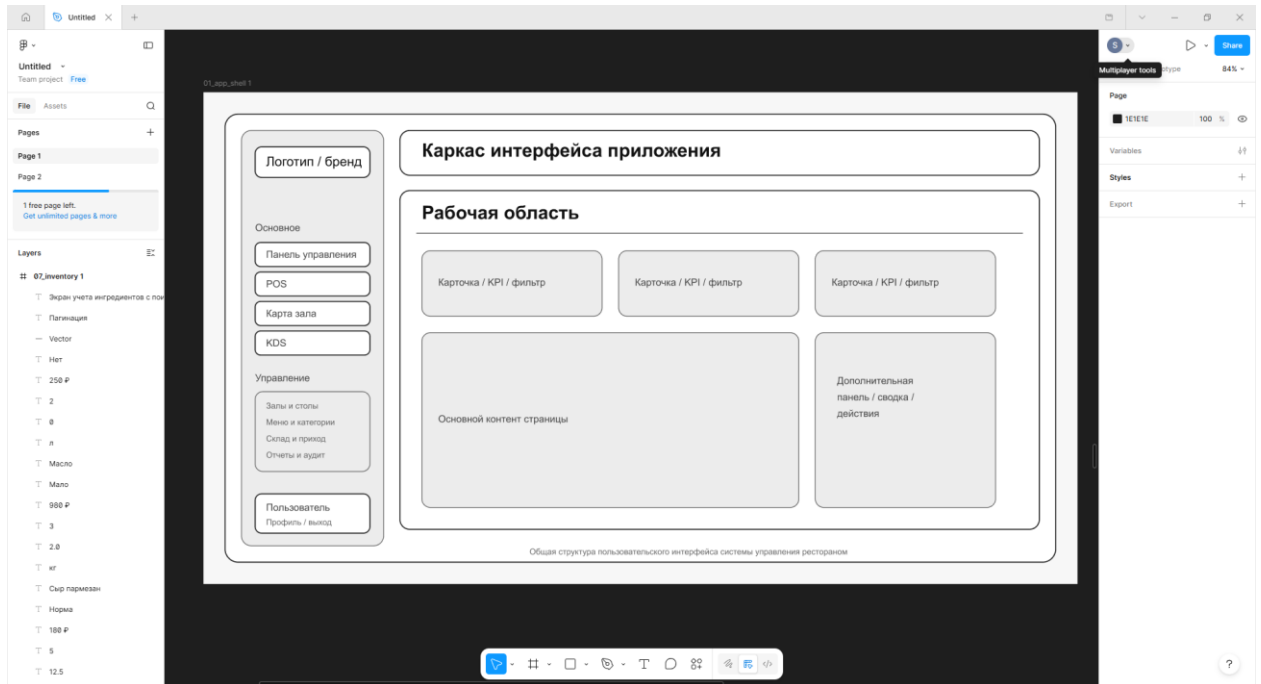


Рисунок 1.1 – Общий каркас интерфейса.

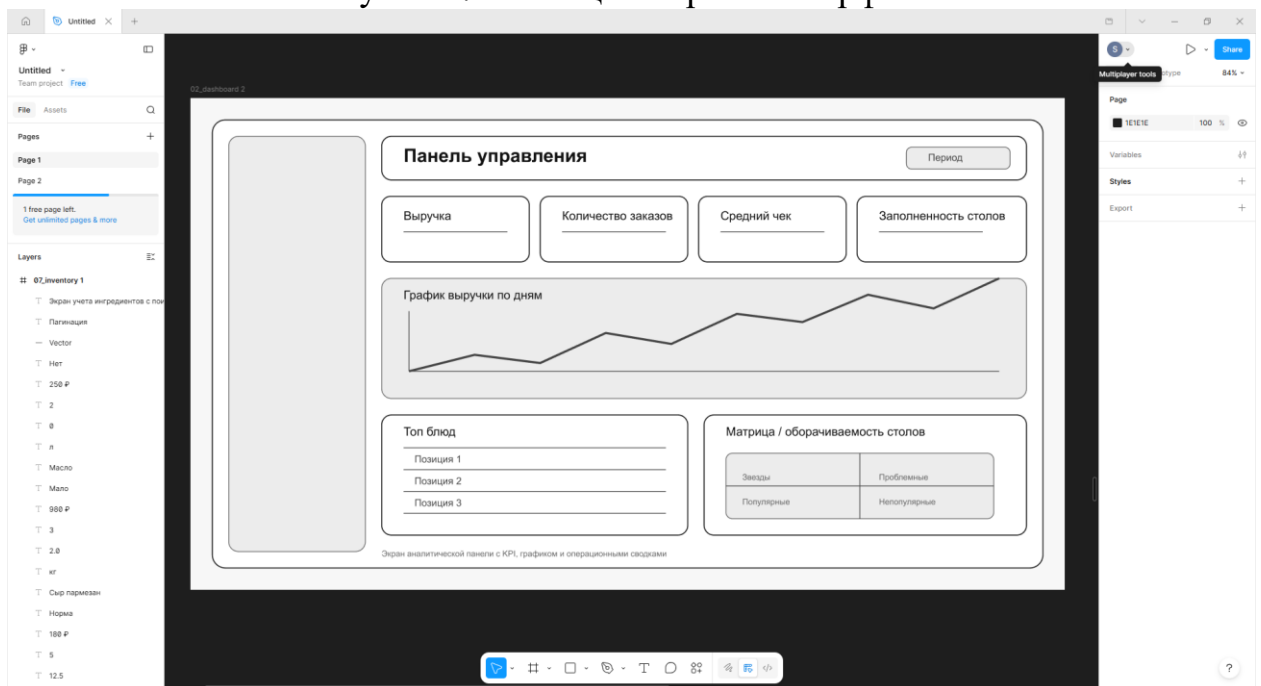


Рисунок 1.2 – Прототип панели управления.

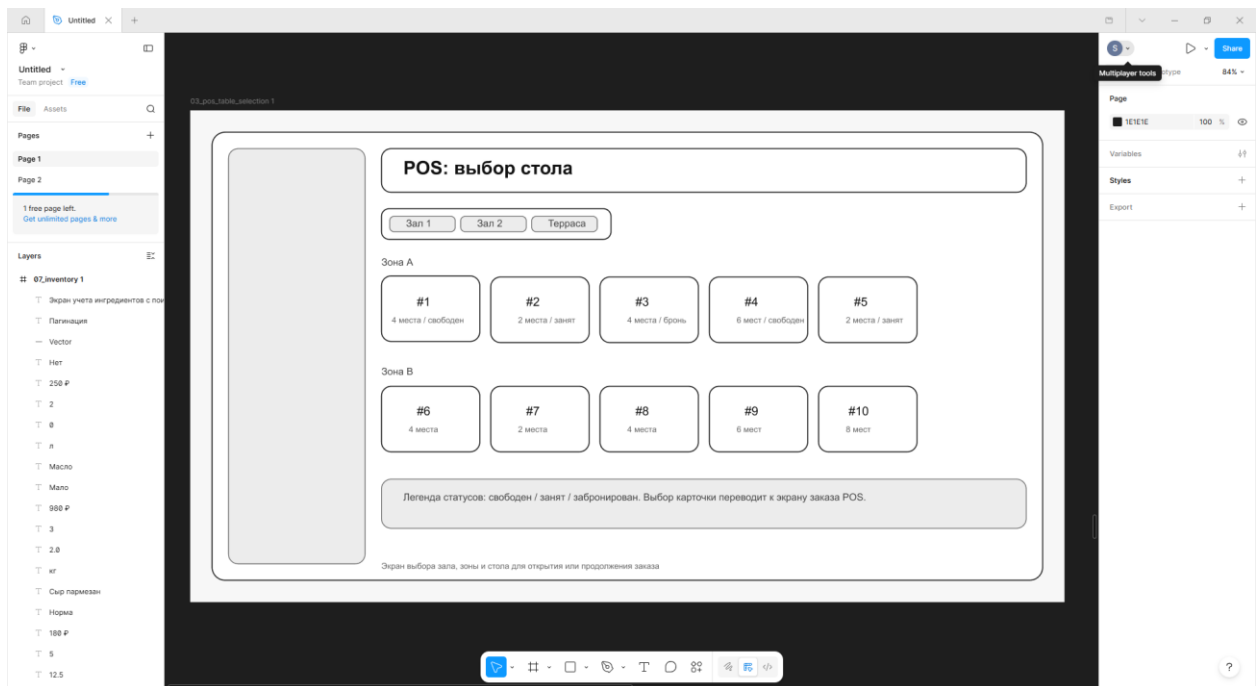


Рисунок 1.3 – POS экран.

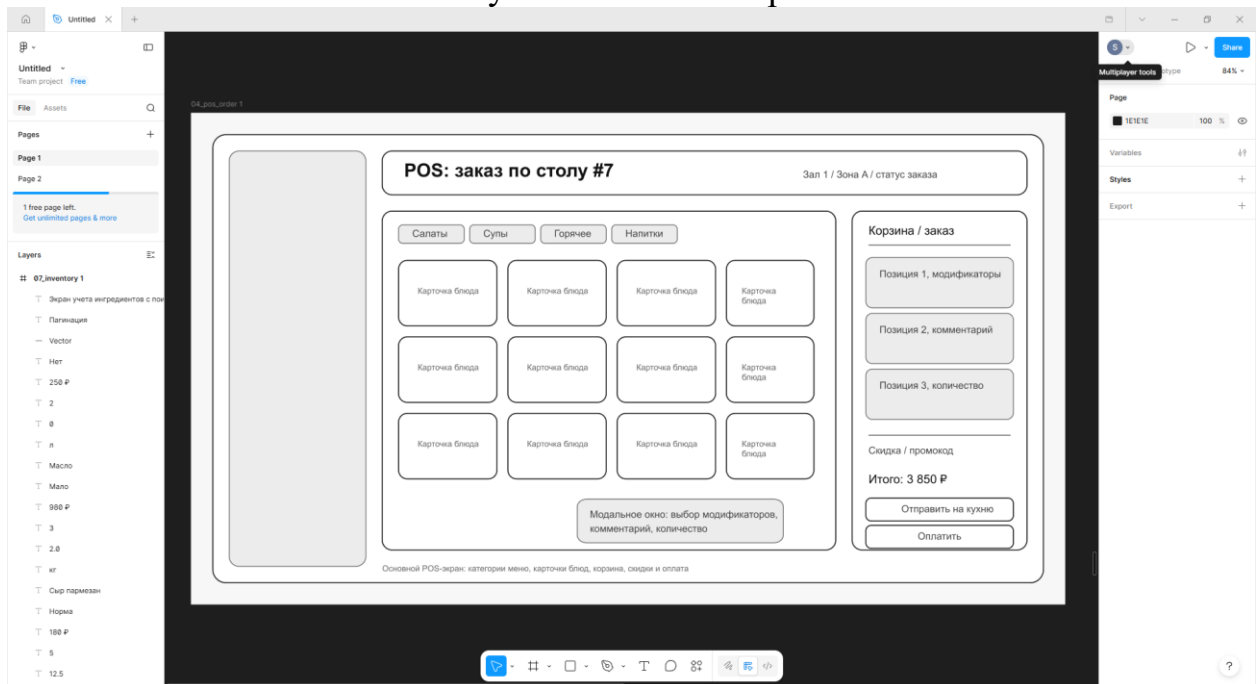


Рисунок 1.4 – POS экран, создание заказа.

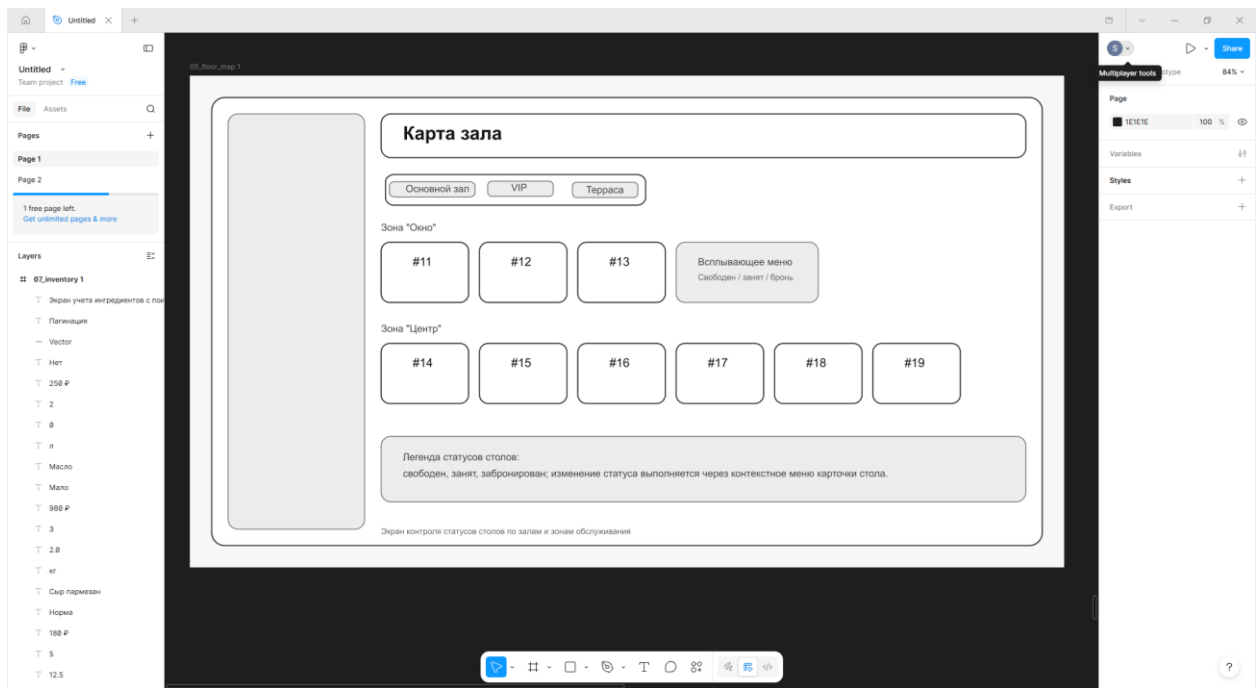


Рисунок 12345 – Экран с картой зала.

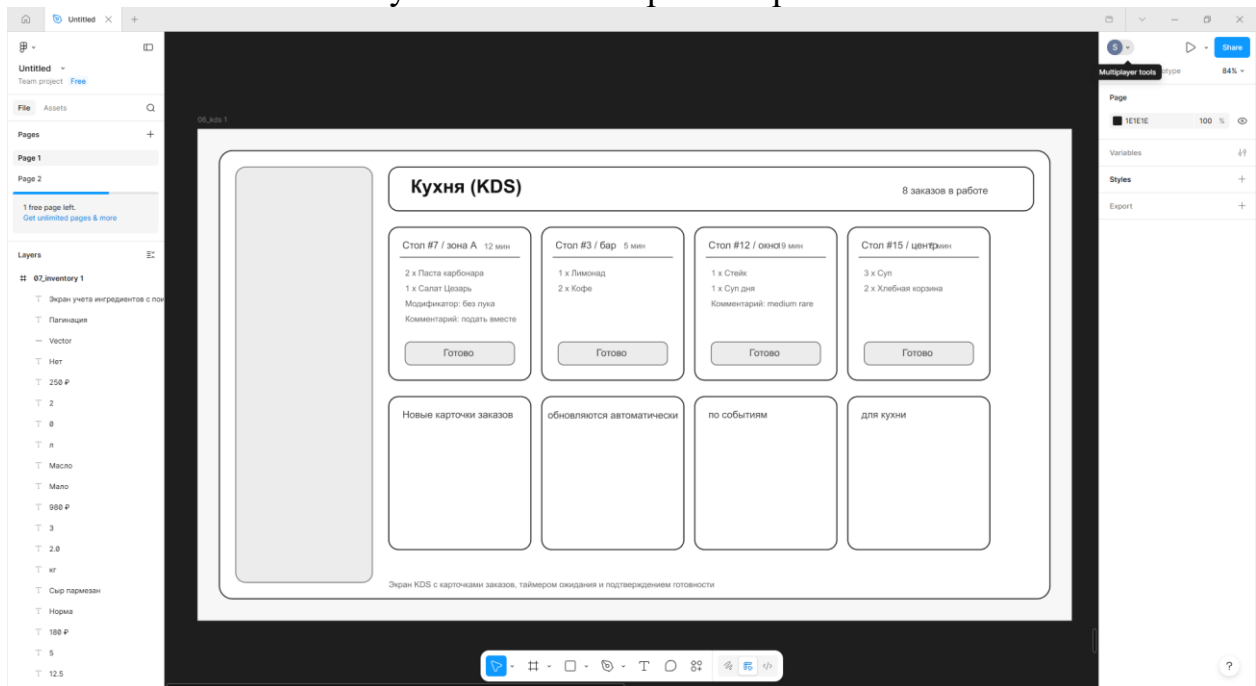


Рисунок 1.4 – Экран кухни. KDS

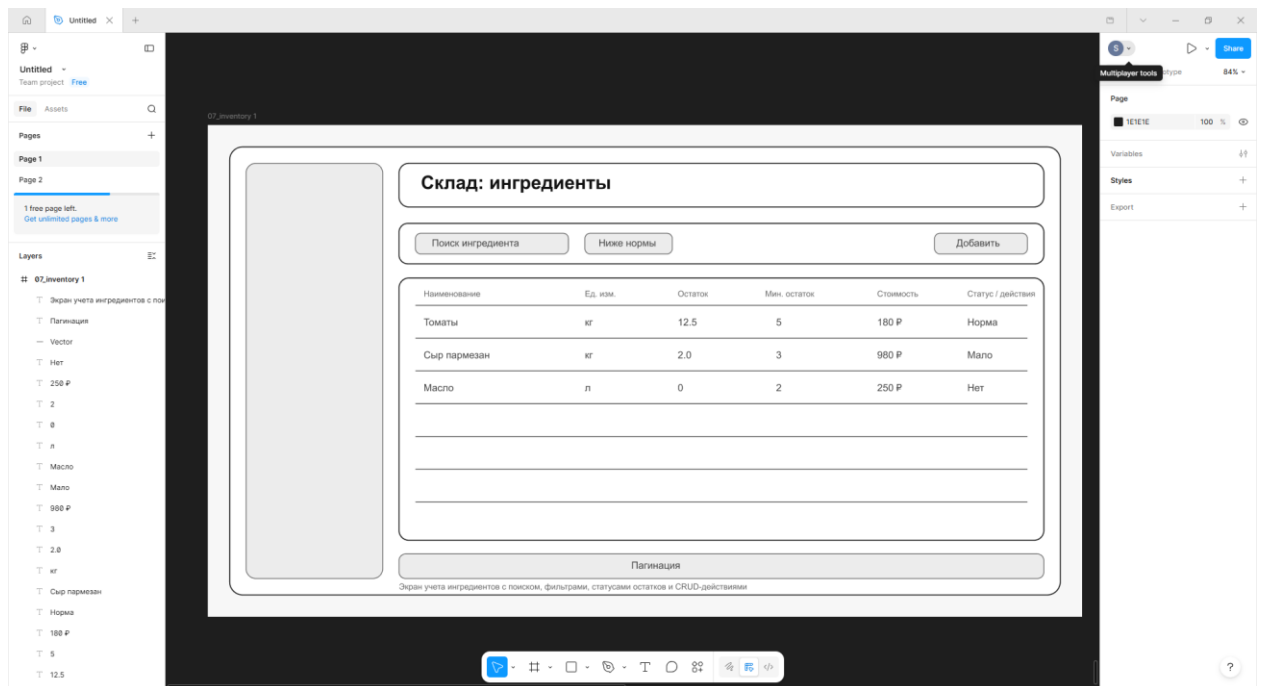


Рисунок 1.3 – Экран склада.

Приложение Г – Выбор технологий. Сравнительные таблицы

Критерий	Laravel (PHP)	Django (Python)	Express.js (Node.js)
Наличие встроенной ORM и поддержки миграций	Да	Да	Нет
Средства аутентификации и авторизации	Да	Да	Ограниченно
Поддержка WebSocket / real-time взаимодействия	Да	Ограниченно	Да
Качество официальной документации	Отличное	Отличное	Хорошее
Наличие готовых пакетов для бизнес-логики и корпоративных задач	Высокое	Среднее	Среднее
Наличие стартовых комплектов и ускорителей разработки	Да	Ограниченно	Практически отсутствует в базовом фреймворке
Средства для очередей и фоновых задач	Да	Ограниченно	Ограниченно

Таблица 4.1. Сравнение серверных технологий.

Критерий	Vue 3 + Inertia.js	React + Next.js	Blade
Возможность создания SPA без отдельного API	Да	Ограниченно	Нет
Реактивность и интерактивность интерфейса	Высокая	Высокая	Низкая
Уровень интеграции с Laravel	Высокий	Средний	Высокий
Поддержка SSR без развёртывания отдельной клиентской инфраструктуры	Да (поддерживается через Inertia SSR)	Ограниченно	Да
Поддержка типизации пропсов и структур данных	Да	Да	Нет

Таблица 4.2. Сравнение технологий для клиентской части

Критерий	PostgreSQL	MySQL	SQLite
Поддержка JSON-полей с возможностью индексации	Высокая (полноценная поддержка JSON/JSONB, развитые механизмы индексации)	Средняя (поддержка JSON имеется, но возможности индексации и работы с данными ограничены)	Низкая
Поддержка ACID-транзакций	Полная	Полная	Полная
Поддержка параллельного выполнения запросов (MVCC)	Высокая	Средняя	Низкая
Поддержка расширений и модулей	Высокая	Ограниченная	Практически отсутствует
Наличие встроенных сложных типов данных (UUID, ENUM, ARRAY и др.)	Высокое	Среднее	Низкое
Поддержка сложных аналитических запросов	Высокая	Средняя	Низкая
Масштабируемость для многопользовательской системы	Высокая	Высокая	Низкая

Таблица 4.3. Сравнение СУБД.

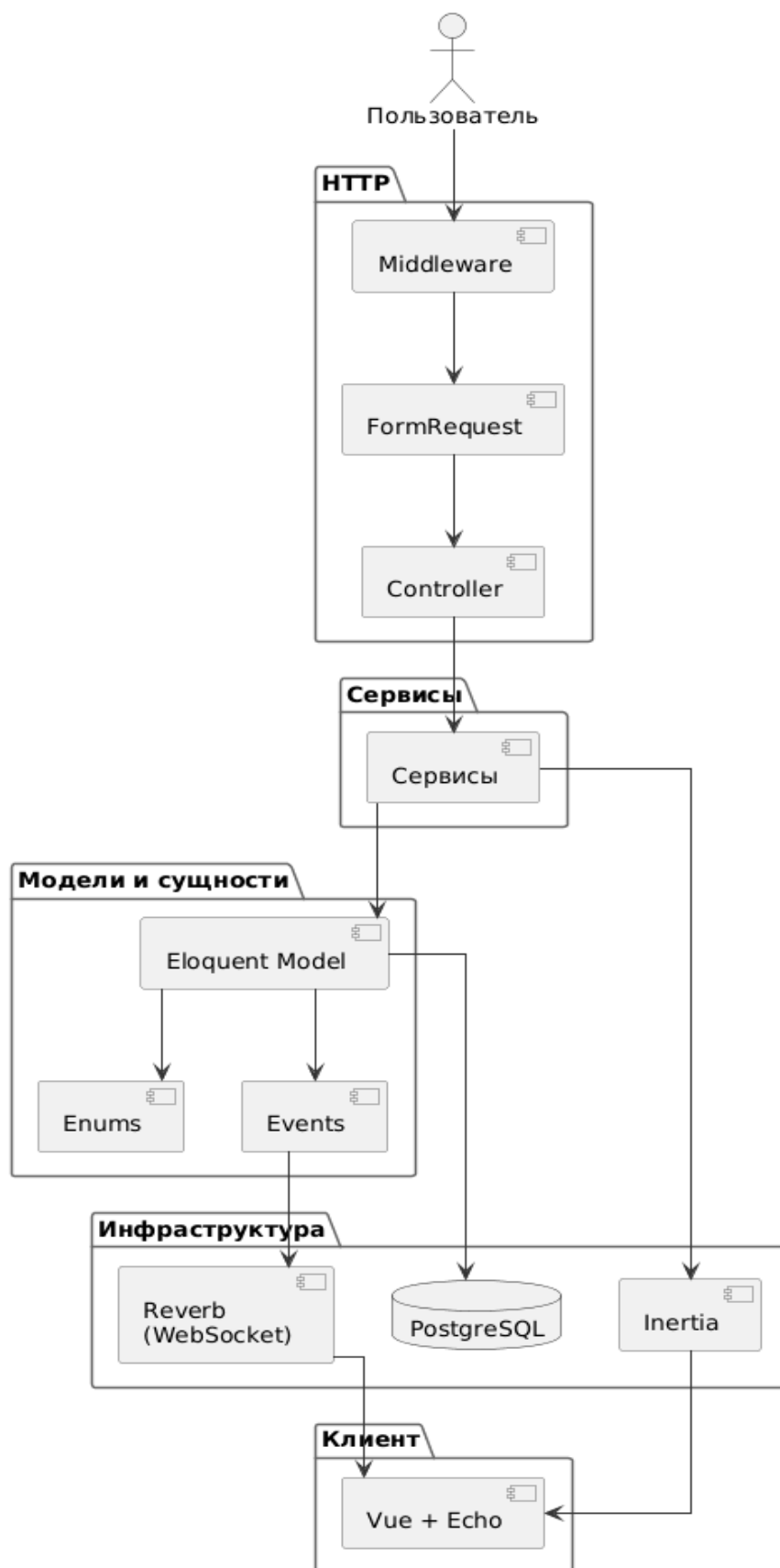


Рисунок 4.1. Диаграмма архитектуры.

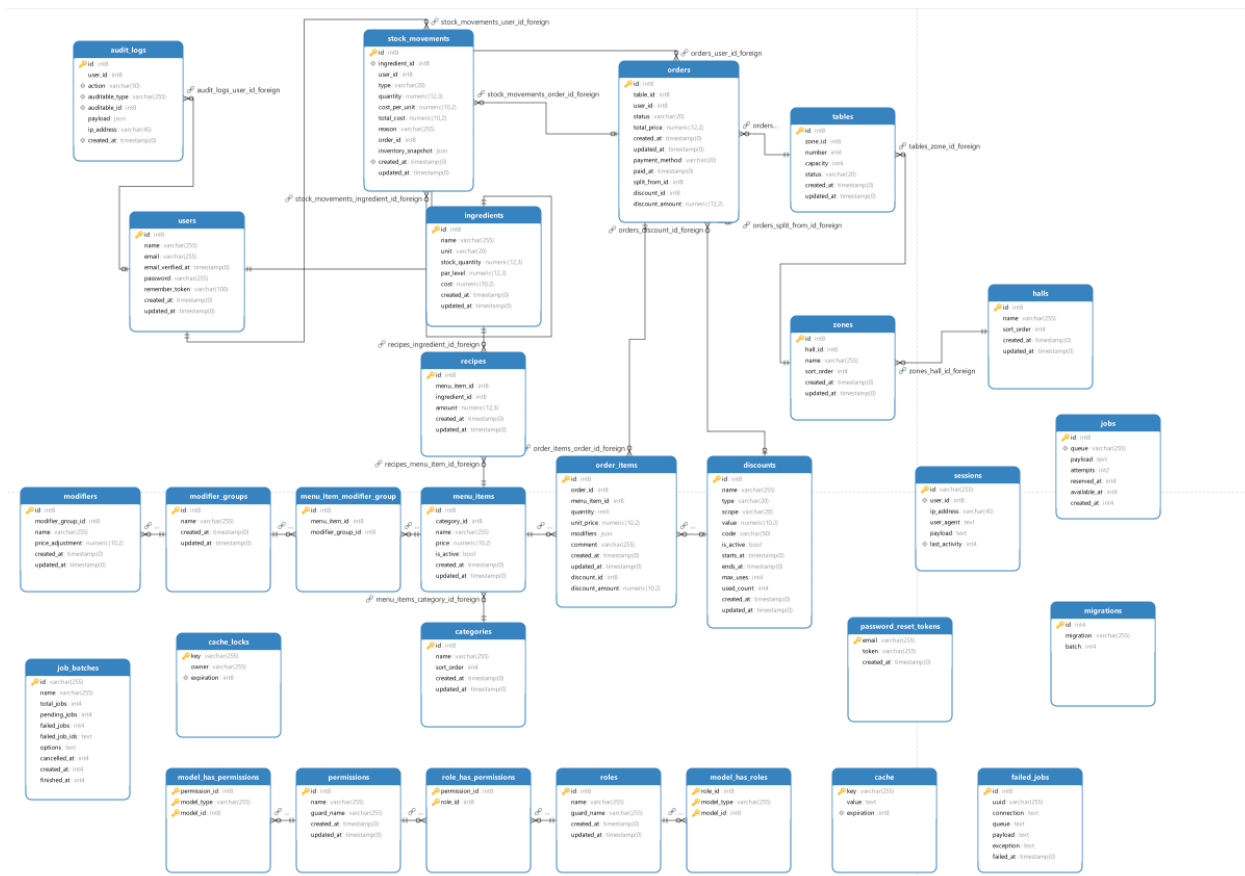


Рисунок 4.2. ER-диаграмма.

Приложение Д – Реализация и тестирование.

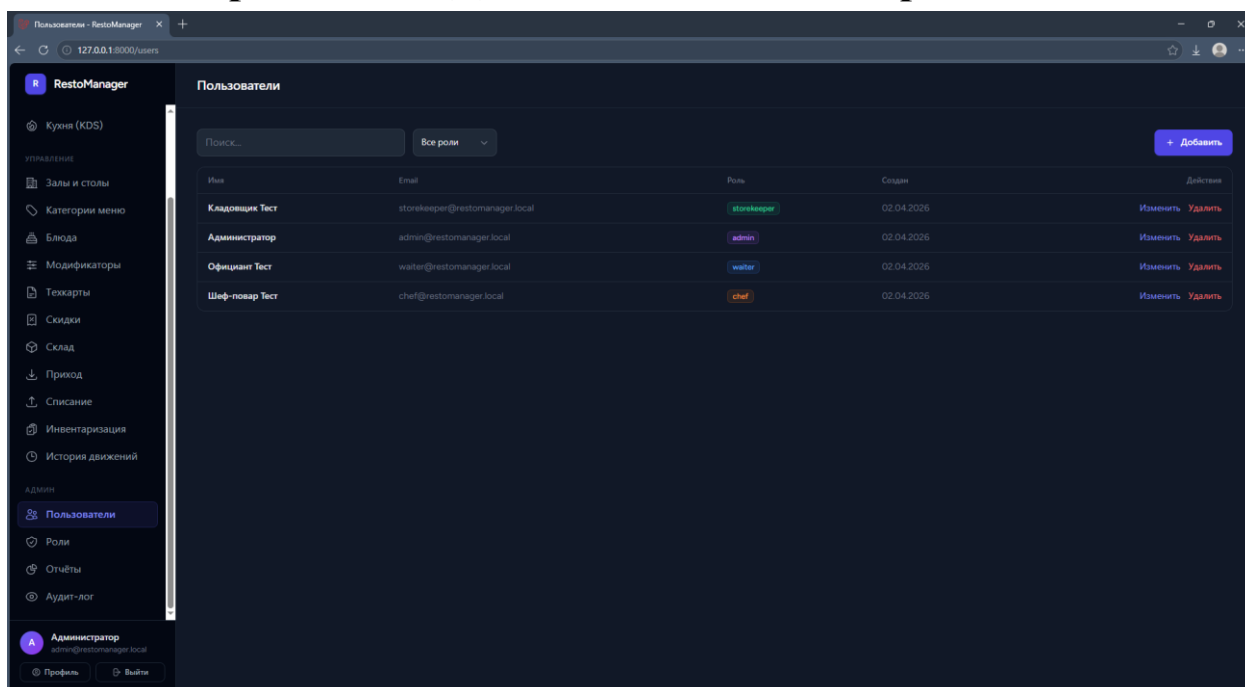


Рисунок 5.1. Страница пользователей.

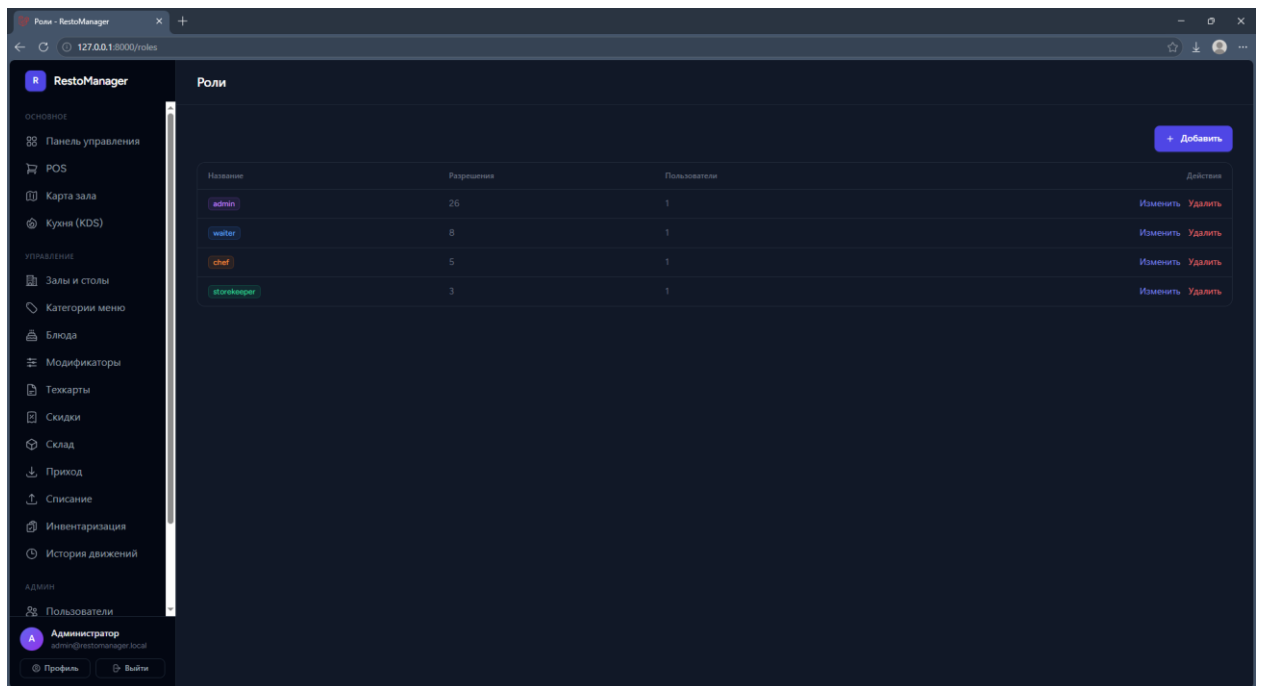


Рисунок 5.2. Страница управления ролями.

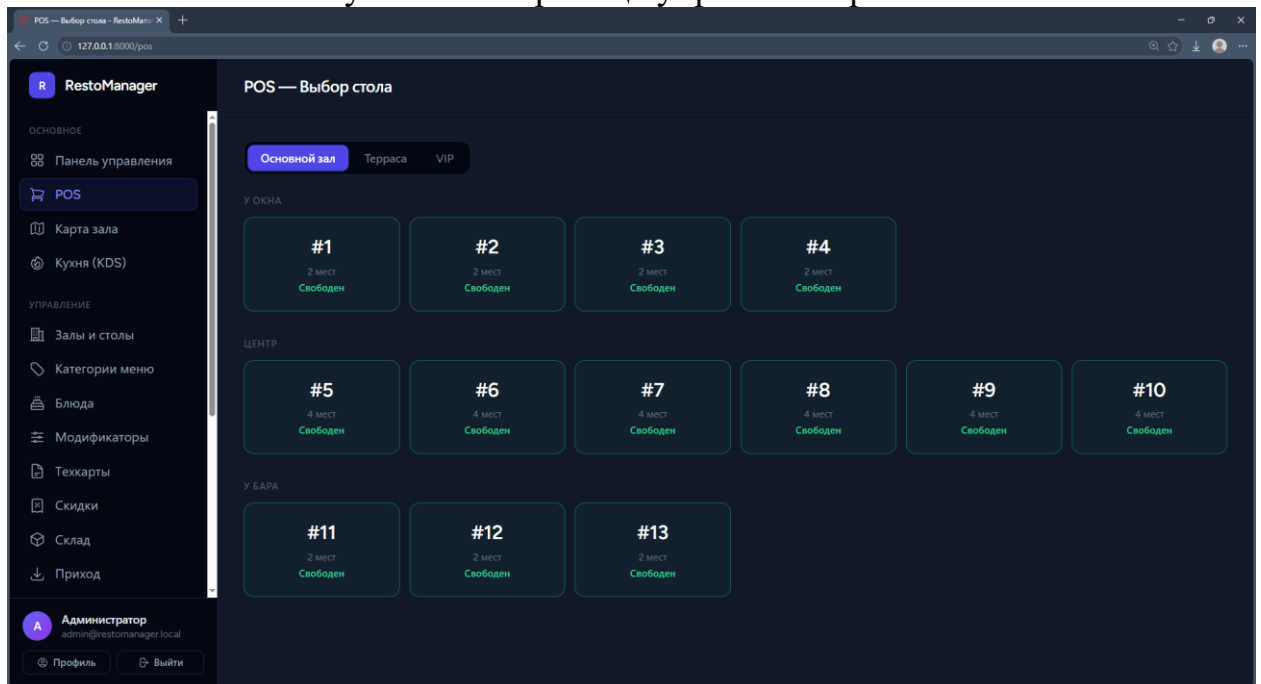
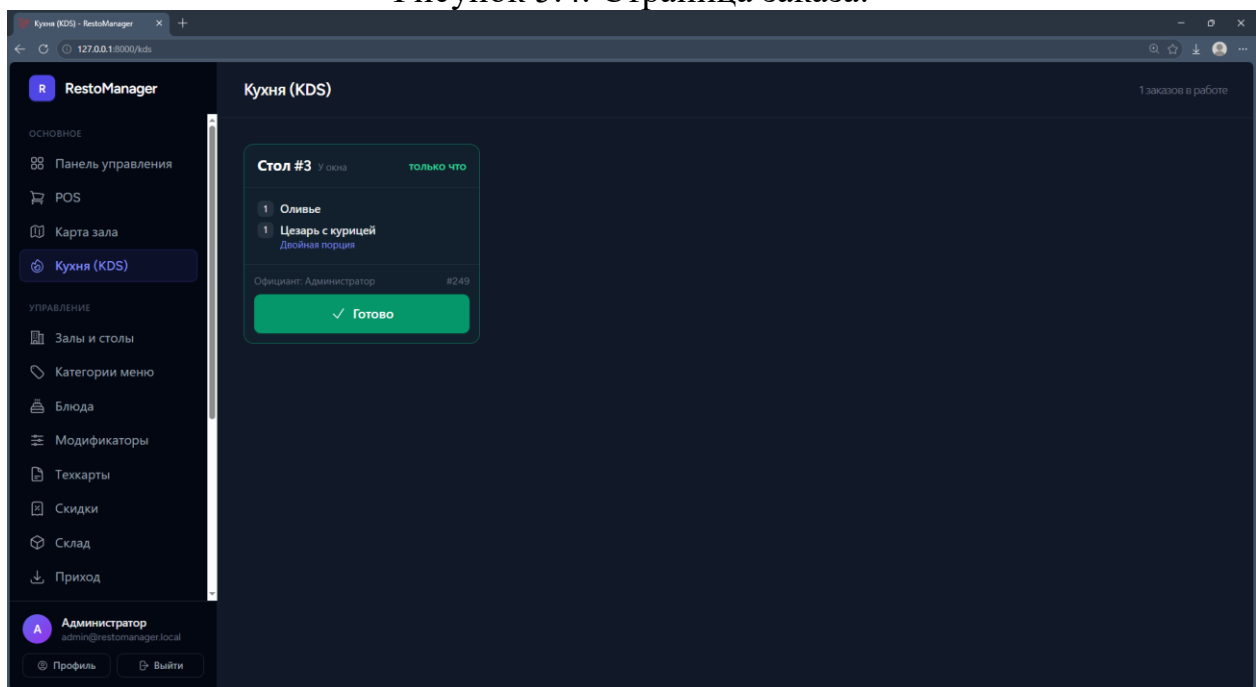
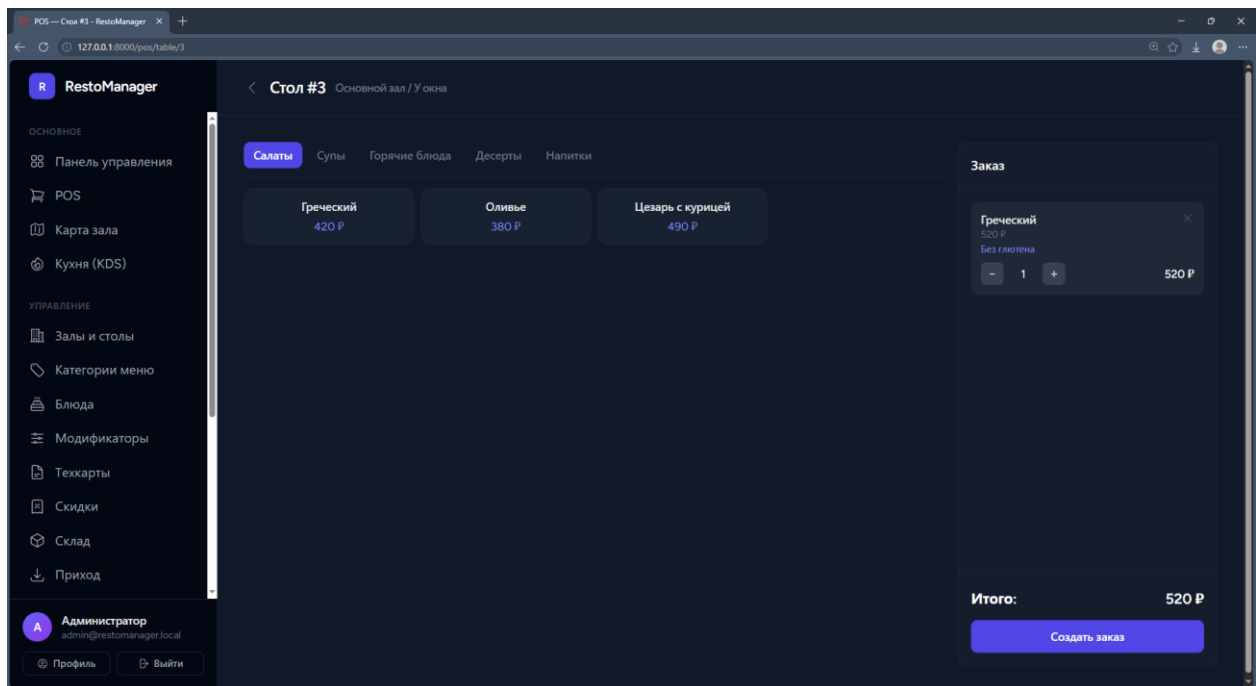


Рисунок 5.3. Страница POS.



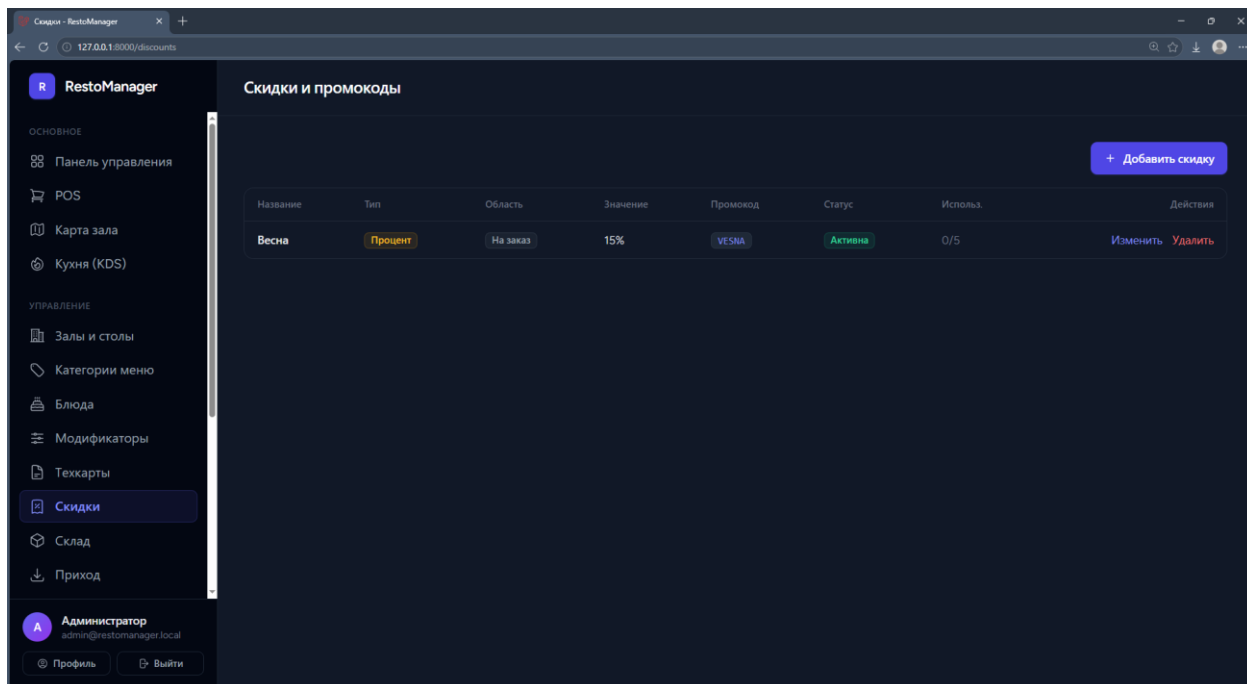


Рисунок 5.6. Страница скидок.

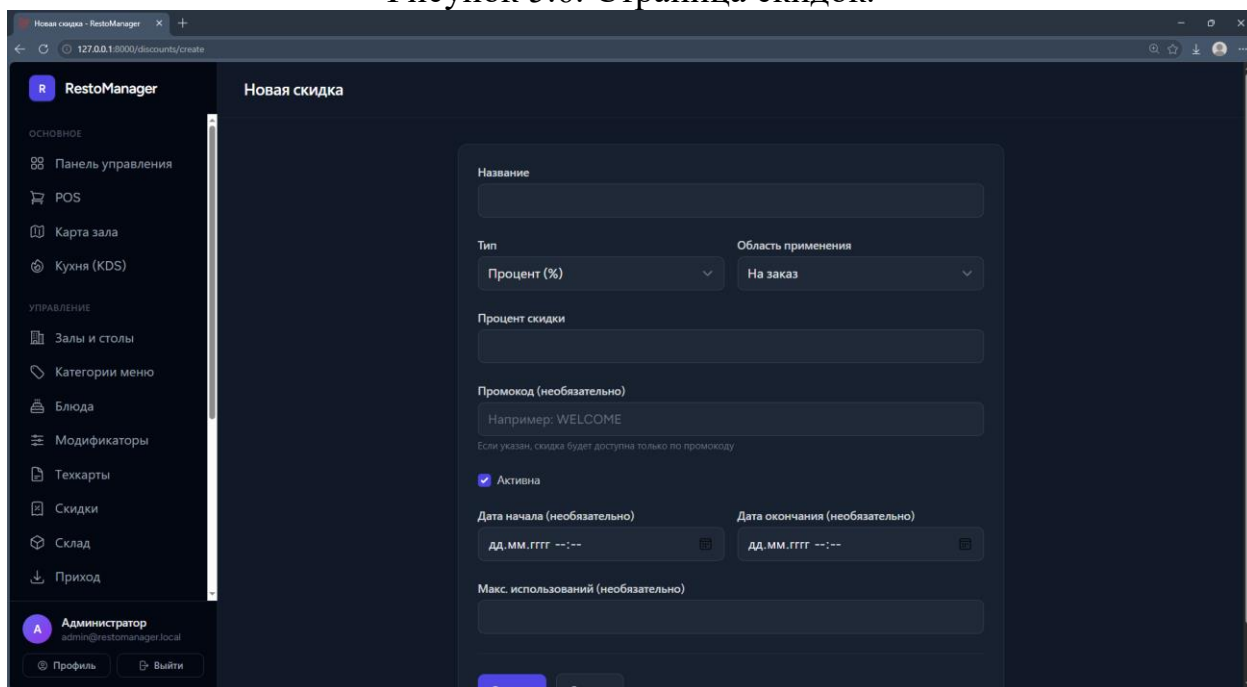


Рисунок 5.7. Форма создания скидки.